FPT UNIVERSITY

Software Architecture and Design — SWD392

Course Project Report

NeuroScan AI

SaaS Platform for MRI/CT Brain Tumor Diagnosis

Subject: SWD392

— Da Nang, April 2025 —

Table of Contents

Note: This Table of Contents is generated via field codes. To ensure page number accuracy after editing, please right-click the TOC and select "Update Field."

Record of Changes

This section tracks all modifications made to this document throughout the project lifecycle. Each entry records the date, type of change (Added / Modified / Deleted), the responsible team member, and a description of the change.

Table 1: Record of Changes

Date	A/M/D	In charge	Change Description
[Please fill in]	[Please fill in]	[Please fill in]	[Please fill in]
[Please fill in]	[Please fill in]	[Please fill in]	[Please fill in]
[Please fill in]	[Please fill in]	[Please fill in]	[Please fill in]
[Please fill in]	[Please fill in]	[Please fill in]	[Please fill in]
[Please fill in]	[Please fill in]	[Please fill in]	[Please fill in]
[Please fill in]	[Please fill in]	[Please fill in]	[Please fill in]
[Please fill in]	[Please fill in]	[Please fill in]	[Please fill in]
[Please fill in]	[Please fill in]	[Please fill in]	[Please fill in]

*A — Added, M — Modified, D — Deleted

I. Overview

This section provides the high-level identification of the NeuroScan AI project — its name, code, group, and software type — along with the project team composition and the member-to-use-case assignment.

I.1 Project Information

Field	Value
Project name	NeuroScan AI
Project code	NSA
Group name	SWD392-G5
Software type	Web App + Mobile App (React Native / Expo) + AI Backend (FastAPI / Python) + SaaS Multi-tenant
Repository	github.com/ntthanh2357/mri (branch: Huyle_MRI/CT)
Tech stack	Node.js 20 + Express 4 + MongoDB 8 (27 models + 2 new) + React Native 0.81 + Expo 54 + FastAPI + TensorFlow 2 + YOLOv8 + Google Gemini
Total use cases	28 (UC-01 → UC-28)
Total BE endpoints	137 (across 15 routers + 2 new)
Total AI endpoints	8 (/predict, /feedback, /approve, /training-stats, /generate_clinical_report, /translate_for_patient, /chat, /summarize_meeting)
Total FE screens	28 (in AppNavigator)
Total diagrams in this report	53 (6 UC + 5 Activity + 28 Sequence + 14 architectural)

I.2 Project Team

The following table lists the six project team members, their roles, and the use cases they are responsible for. Use case assignment follows each member's domain ownership: patient-facing, doctor-facing, AI/ML, scheduling, pharmacy, or authentication/billing.

Table 2: Project Team & UC Assignment

Full Name	Role	Email	Mobile	Assigned UCs
Nguyen Tuan Thanh	Patient module	<<thanhnt@fpt.edu.vn>>	<<phone>>	UC-01, UC-03, UC-06, UC-07, UC-08, UC-10
Le Hai Nam	Doctor module	<<namlh@fpt.edu.vn>>	<<phone>>	UC-13, UC-14, UC-16, UC-17
Nguyen Chau Quang	Scheduling module	<<quangnc@fpt.edu.vn>>	<<phone>>	UC-12, UC-19, UC-20, UC-21
Le Tran Gia Huy	AI/MRI module	<<huyltg@fpt.edu.vn>>	<<phone>>	UC-09, UC-15
Le Van Minh	Admin/Pharmacy	<<minhlv@fpt.edu.vn>>	<<phone>>	UC-22, UC-24, UC-25, UC-26, UC-27, UC-28
Nguyen Van Hoang	Auth & Billing	<<hoangnv@fpt.edu.vn>>	<<phone>>	UC-02, UC-04, UC-05, UC-11, UC-18, UC-23

Lecturer: Nguyen Trung Kien — kiennt@fe.edu.vn — 0912656836

II. Requirement Specification

II.1 Problem Description

NeuroScan AI is a multi-tenant Software-as-a-Service (SaaS) platform that enables Vietnamese hospitals to perform AI-assisted diagnosis of brain tumors from MRI/CT scans. The platform replaces the current manual, paper-based workflow with a unified digital pipeline combining a Bayesian ensemble of three Convolutional Neural Networks (ResNet50 w=0.8, EfficientNet w=0.1 with temperature scaling T=1.3, DenseNet w=0.1 with T=1.3), YOLOv8 tumor localization, and Google Gemini Vision as a cross-validation arbiter when the CNN ensemble and YOLO disagree. The system is deployed across multiple hospitals with strict data isolation: every clinical record is automatically scoped by hospitalId at the MongoDB query layer using AsyncLocalStorage + a Mongoose plugin, ensuring no patient data ever leaks across tenants. The platform also exposes four LLM agents — a radiology report writer, a patient-friendly translator, a RAG chatbot grounded in Vietnamese clinical protocols (QD 1514), and a medical secretary that summarizes multi-doctor consultations — to support the entire clinical workflow from admission to discharge. HIPAA-grade audit logging, OTP-based password reset, and Google/Zalo Single Sign-On (SSO) are first-class features.

II.2 Major Features

1. FE-01: Run AI-assisted brain tumor diagnosis from MRI/CT scans using a Bayesian ensemble of three CNNs with Test-Time Augmentation, YOLOv8 localization, and Grad-CAM heatmap visualization.
2. FE-02: Cross-validate CNN and YOLO predictions with Google Gemini Vision as an arbiter, returning a structured JSON verdict with confidence and reasoning when the two models disagree.
3. FE-03: Multi-tenant SaaS architecture — each hospital has isolated data, separate Google Drive folder structure, and independent subscription/pricing configuration.
4. FE-04: Patient authentication via Google SSO, Zalo SSO, email/password, phone+OTP, and Firebase Authentication — with hospital staff activation flow using temporary passwords.
5. FE-05: Full Electronic Medical Record (EMR) — paper-form digitization of the Vietnamese Ministry of Health brain-cancer medical record, with version-controlled edits, care sheets, consultations, and signed consent forms.
6. FE-06: Laboratory Information System (LIS) integration — receive biomarker results via barcode, validate against gendered reference ranges, flag abnormal HIGH/LOW values.

7. FE-07: Pharmacy management with prescription safety checker — psychotropic classification, drug-drug interaction matrix, BMI-based corticosteroid warning, Gadolinium allergy ADR check.
8. FE-08: Hospital staff scheduling — weekly grid view, shift swap requests with admin approval, conflict prevention against active visits.
9. FE-09: Billing & invoicing — auto-generate invoices when visit completes, decrement drug stock, mark prescriptions as billed, support pending invoices for later checkout.
10. FE-10: Polling-based notification system (every 20 seconds) — list notifications for doctor/nurse/technician on visit creation, MRI order, AI result ready, low-stock alerts.
11. FE-11: Hospital onboarding workflow — system admin provisions a hospital (generates BV_XXX code, temp credentials, Google Drive structure), hospital admin completes onboarding form and uploads license file.
12. FE-12: System admin SaaS backoffice — manage tenants, subscriptions, AI model versions, backup/restore per hospital, anonymize audit logs (HIPAA), revenue/drug reports with CSV export.
13. FE-13: Four LLM agents — clinical report writer, patient-friendly translator, RAG chatbot (grounded in 6 Vietnamese clinical protocol PDFs), and medical secretary that summarizes multi-doctor consultations.
14. FE-14: Patient Premium plan with VNPay/MoMo payment integration — Subscription model tracks plan/status/expiresAt, webhook updates status on payment success.
15. FE-15: Medicine Reminder system — auto-generate reminders per drug × day × time slot when prescription is created (BR-06), patient marks reminders as done throughout the day.

II.3 Context Diagram

The context diagram below presents NeuroScan AI as a single black box and identifies the external entities — both human actors and external software systems — that interface with it. Data flows are labeled with the specific information exchanged.

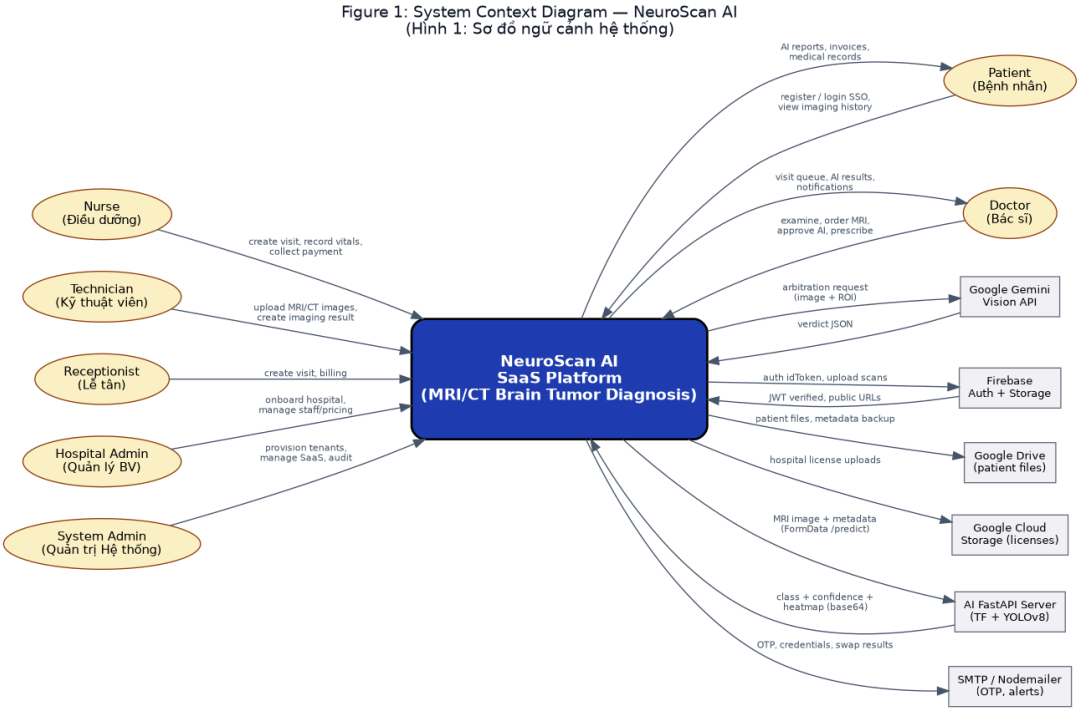

Figure 1: System Context Diagram

II.4 Nonfunctional Requirements

The following table lists the nonfunctional requirements (NFRs) that the system must satisfy. These are properties of the system that are not direct user-facing features but critically affect usability, security, and operational quality.

Table 3: Nonfunctional Requirements

#	Feature	System Function	Description
NFR-01	Performance	AI Inference Latency	BayTTA ensemble + YOLO + (optional) Gemini arbitration must return within 8 seconds on CPU; <3s on GPU. API endpoints respond <500ms (p95).
NFR-02	Security / HIPAA	Patient Data Protection	JWT with tokenVersion for global logout; phone HMAC-hashed with PHONE_SALT; patient_id SHA-256 hashed and patient_name masked before sending to Gemini; OTP TTL 5 min; audit log anonymization.
NFR-03	Multi-tenancy Isolation	Hospital Data Segregation	AsyncLocalStorage + Mongoose plugin auto-injects hospitalId into all queries of tenancy-enabled models; defence-in-depth manual checks in controllers; per-hospital Google Drive folder structure.

#	Feature	System Function	Description
NFR-04	Scalability	Horizontal Scale-out	Stateless BE (Express) and AI (FastAPI) servers horizontally scalable behind load balancer; MongoDB sharded by hospitalId; Redis for notification cache and rate-limit.
NFR-05	Availability	Service Uptime	Graceful degradation: GCS → local /uploads fallback, Google Drive → GCS fallback, Gemini 429 → keep ensemble prediction, Firebase key missing → warn (no crash). Target uptime 99.5%.
NFR-06	Maintainability	Layered Codebase	Strict separation: routes → controllers → services → models; FE: screens → services → utils; AI: main.py → preprocess.py + localization.py. 27 Mongoose schemas + 2 new, 13 controllers + 2 new, 5 services.
NFR-07	Usability	Vietnamese-first UI	All status enums, payment methods, biomarker categories, drug categories in Vietnamese; 15-min inactivity auto-logout (web); 28 navigation routes role-based.

#	Feature	System Function	Description
NFR-08	Auditability	EMR Version Control + Audit Log	Every MedicalRecord edit creates an EMRVersion diff entry; AuditLog records all admin/auth/hospital/dataset operations with performedBy + hospitalId + timestamp.

II.5 Functional Requirements

II.5.1 Actors

An actor is a person or external system that interacts with the system to perform a use case. The table below describes the seven actors identified for NeuroScan AI.

Table 4: System Actors

#	Actor	Description
1	Patient	Registers via SSO or email; books visits; views imaging history; fills brain-cancer medical record form; pays invoices; upgrades to Premium for AI priority; marks medication reminders as taken.
2	Doctor	Examines patients; orders MRI/CT scans; reviews AI results; approves or corrects AI predictions with bounding box feedback; edits findings/conclusion; prescribes drugs (auto-generates reminders per BR-06).
3	Nurse	Creates visits; records vital signs (pulse, BP, SpO2, temperature); creates pending invoices; collects payments; writes care sheets; updates drug stock.
4	Technician	Performs MRI/CT scans; uploads scan images (base64); creates imaging results with DICOM metadata; notifies ordering doctor when AI-ready.
5	Receptionist	Creates walk-in visits; manages billing queue; processes pending invoice payments; collects cash or transfer.

#	Actor	Description
6	Hospital Admin	Completes hospital onboarding form (name, tax code, address, legal rep, IT contact, license upload); manages staff (create/lock/unlock); sets pricing; manages drug inventory; views financial reports.
7	System Admin	SaaS operator — provisions new hospitals; activates/locks hospitals; manages subscriptions; backs up / restores per-hospital data; manages AI model versions; views audit logs and anonymizes them; verifies tenant isolation.

II.5.2 Use Cases

The system has 28 use cases (UC-01 to UC-28). Six use case diagrams are provided below, grouped by module to keep each diagram readable: Common/Authentication, Patient Portal, Clinical Workflow, Staff Scheduling, Hospital Onboarding, and Admin Backoffice.

II.5.2.1 Use Case Diagrams

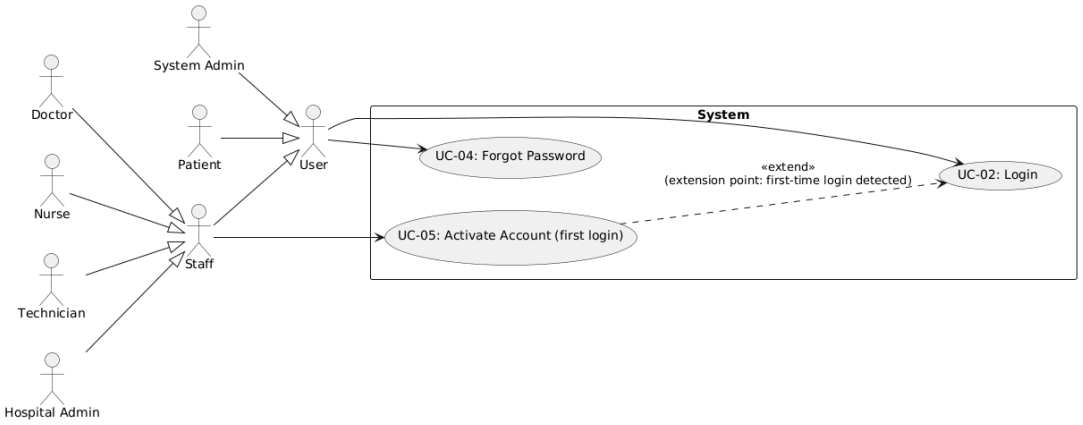

Figure 2: UC Diagram 1 — Common / Authentication

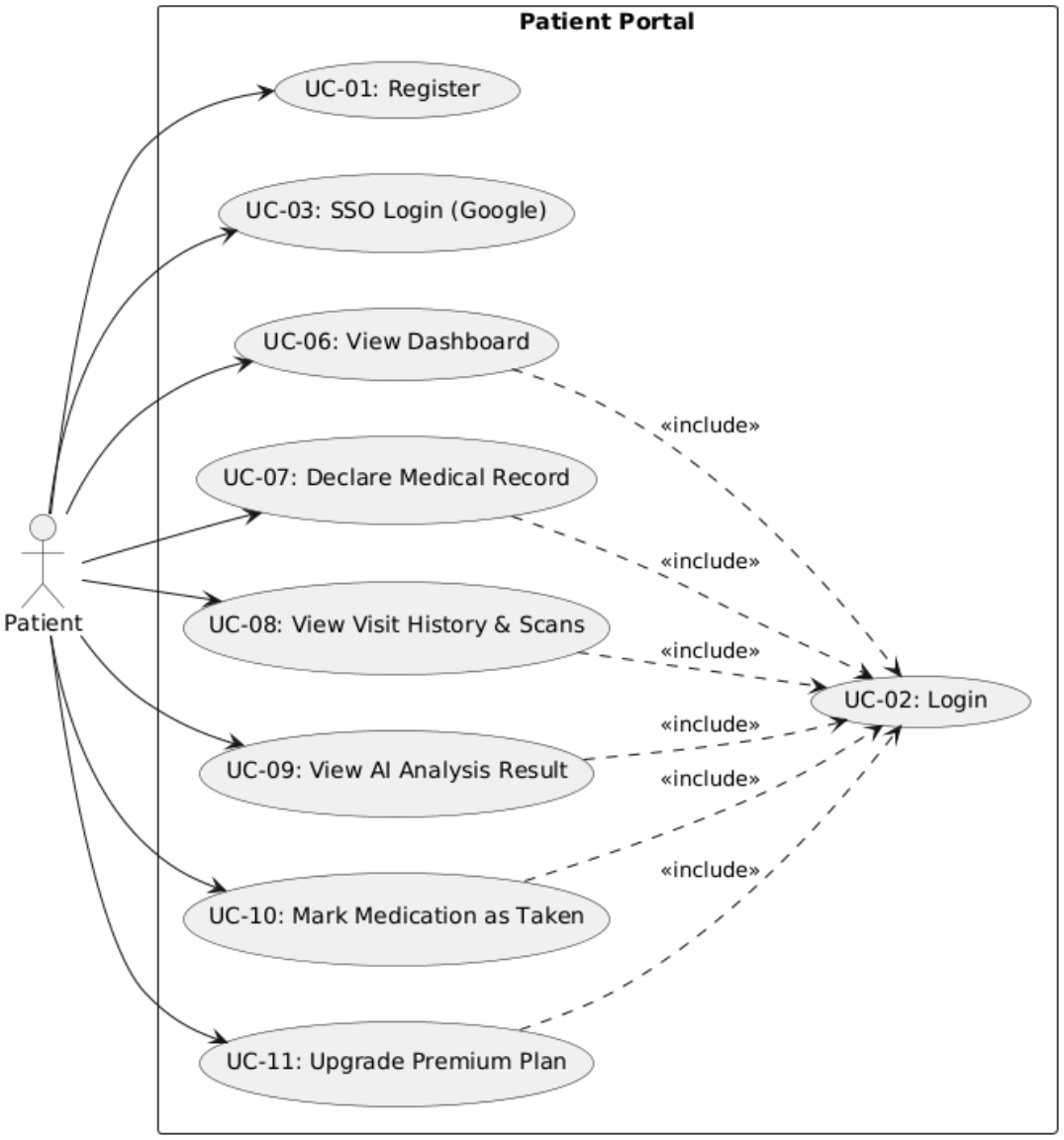

Figure 3: UC Diagram 2 — Patient Portal

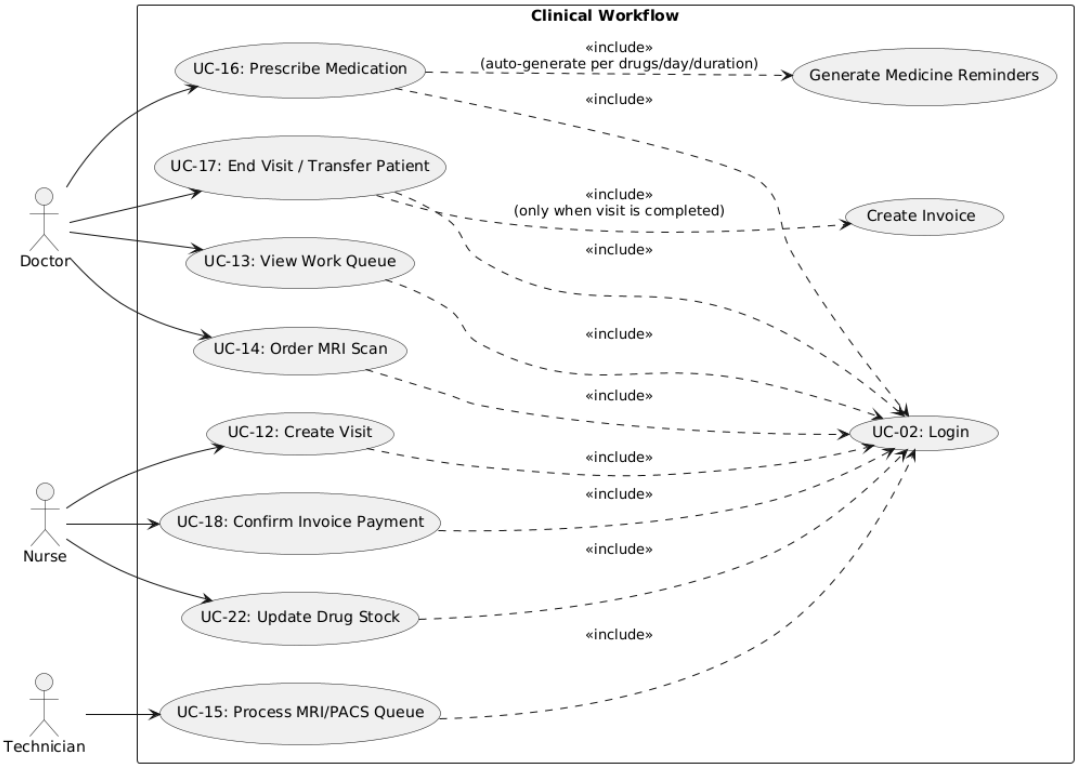

Figure 4: UC Diagram 3 — Clinical Workflow

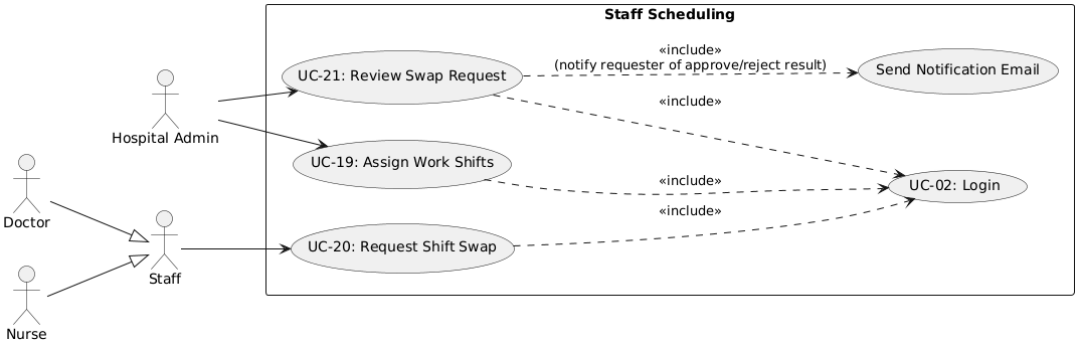

Figure 5: UC Diagram 4 — Staff Scheduling

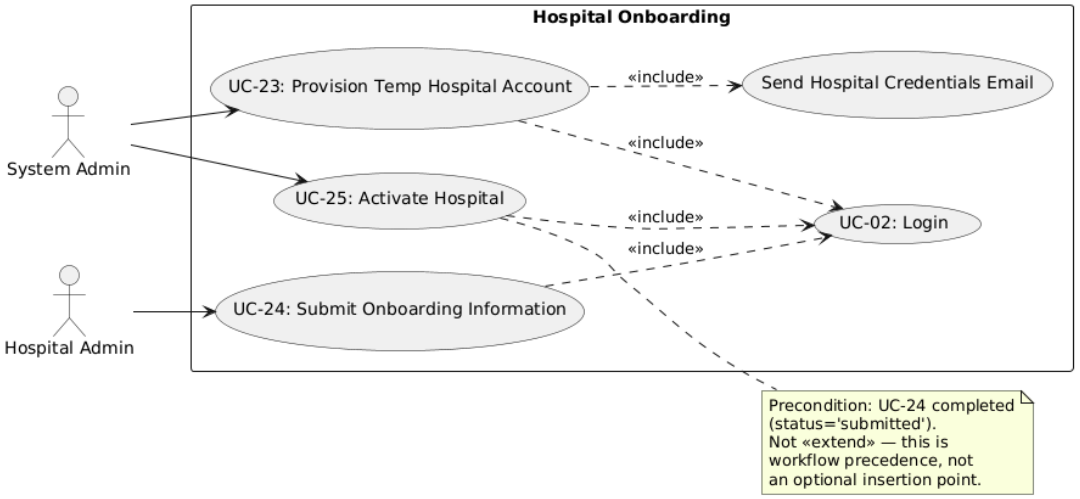

Figure 6: UC Diagram 5 — Hospital Onboarding

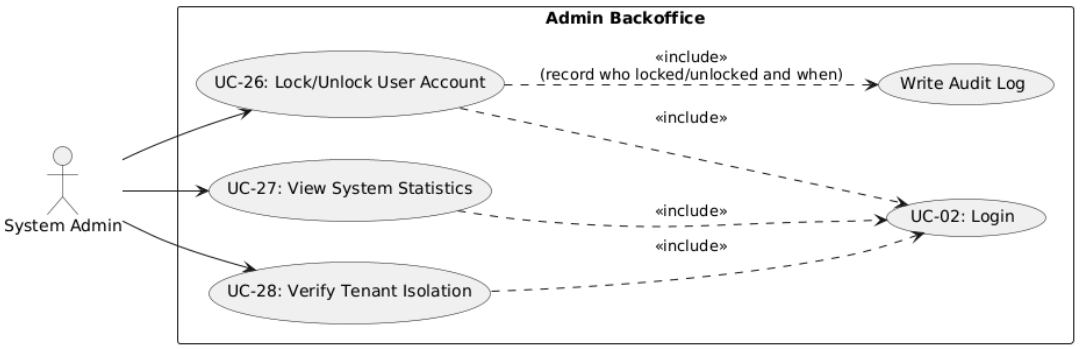

Figure 7: UC Diagram 6 — Admin Backoffice

II.5.2.2 Use Case Catalog

The table below summarizes all 28 use cases. Each use case is assigned to a specific team member based on their module ownership. Detailed Functional Descriptions follow in II.5.2.4.

Table 5: Use Case Catalog with Member Assignment

ID	Use Case	Primary Actor	Member
UC-01	Register Patient Account	Patient	Nguyen Tuan Thanh
UC-02	Login	User (any role)	Nguyen Van Hoang
UC-03	SSO Login (Google)	Patient	Nguyen Tuan Thanh
UC-04	Forgot Password	User (any role)	Nguyen Van Hoang
UC-05	Activate Account (first login)	Staff (Doctor/Nurse/Technician/Hospital Admin)	Nguyen Van Hoang
UC-06	View Patient Dashboard	Patient	Nguyen Tuan Thanh
UC-07	Declare Medical Record	Patient	Nguyen Tuan Thanh
UC-08	View Visit History and MRI/CT Scans	Patient	Nguyen Tuan Thanh
UC-09	View AI Analysis Result	Patient	Le Tran Gia Huy
UC-10	Mark Medication as Taken	Patient	Nguyen Tuan Thanh
UC-11	Upgrade to Premium Plan	Patient	Nguyen Van Hoang
UC-12	Create Visit	Nurse	Nguyen Chau Quang
UC-13	View Work Queue	Doctor	Le Hai Nam
UC-14	Order MRI Scan	Doctor	Le Hai Nam
UC-15	Process MRI / PACS Queue	Technician	Le Tran Gia Huy
UC-16	Prescribe Medication (Auto-generate Reminders)	Doctor	Le Hai Nam

ID	Use Case	Primary Actor	Member
UC-17	End Visit / Transfer Patient	Doctor	Le Hai Nam
UC-18	Confirm Invoice Payment	Nurse / Cashier	Nguyen Van Hoang
UC-19	Assign Work Shifts	Hospital Admin	Nguyen Chau Quang
UC-20	Request Shift Swap	Staff (Doctor / Nurse / Technician)	Nguyen Chau Quang
UC-21	Review Swap Request	Hospital Admin	Nguyen Chau Quang
UC-22	Update Drug Stock	Nurse / Hospital Admin	Le Van Minh
UC-23	Provision Temp Hospital Account	System Admin	Nguyen Van Hoang
UC-24	Submit Onboarding Information	Hospital Admin	Le Van Minh
UC-25	Activate Hospital	System Admin	Le Van Minh
UC-26	Lock / Unlock User Account	System Admin	Le Van Minh
UC-27	View System Statistics	System Admin	Le Van Minh
UC-28	Verify Tenant Isolation	System Admin	Le Van Minh

II.5.2.3 Business Rules

The following 7 business rules are referenced by use cases throughout this document. Each rule has a unique ID so use cases can reference it without repeating the rule text.

ID	Business Rule	Description
BR-01	Password Encoding	Password is hashed with bcrypt (10 salt rounds) before being stored
BR-02	Account Lockout	An account with isLocked=true cannot log in — auth.middleware rejects with 403

ID	Business Rule	Description
BR-03	Duplicate Shift Prevention	A staff member cannot be assigned the same shift type twice on the same day — unique index {staffId, date, shift}
BR-04	Hospital Subscription Gate	When the hospital subscription is expired / suspended, staff (except admin) cannot log in — auth.middleware checks subscriptionStatus
BR-05	Cross-Hospital Referral	Patients can be viewed cross-hospital to support transfers (checkPatientTenancy helper), but staff assigned to a Visit must be in the same hospital as the Visit
BR-06	Reminder Time Slots	Reminder times are fixed by frequency per day: 1→08:00; 2→08:00,20:00; 3→08:00,13:00,20:00; 4→08:00,12:00,17:00,21:00 — REMINDER_TIME_SLOTS map in reminder.controller.js
BR-07	Onboarding Sequence	A hospital must reach status='submitted' (UC-24) before the System Admin can activate it (UC-25) — mandatory business sequence, not an extend relationship

II.5.2.4 Detailed Use Case Descriptions

All 28 use cases are described below using the Functional Description Template. Each use case is marked with the assigned team member.

UC-01 — Register Patient Account

Assigned member: Nguyen Tuan Thanh

UC ID & Name	UC-01 — Register Patient Account
Assigned to	Nguyen Tuan Thanh
Primary Actor	Patient
Secondary Actors	none
Trigger	Patient taps "Register" on the Welcome/Login screen
Description	Patient self-registers an account to use the B2C portal
Preconditions	PRE-1: Email has not been previously registered
Postconditions	POST-1: New User created with role='patient'; POST-2: Patient can log in immediately
Normal Flow	1.0.1 Patient enters email, password, full name1.0.2 System checks email does not already exist1.0.3 System hashes password (bcrypt), creates User1.0.4 System redirects to the Login screen
Alternative Flows	—
Exceptions	1.0.E1 Email already exists → show error "Email already in use", keep form intact
Priority	Must Have
Frequency of Use	Once per new patient
Business Rules	BR-01 (Password Encoding)
Other Information	—
Assumptions	Patient has access to the entered email inbox

UC-02 — Login

Assigned member: Nguyen Van Hoang

UC ID & Name	UC-02 — Login
Assigned to	Nguyen Van Hoang
Primary Actor	User (any role)
Secondary Actors	none
Trigger	User enters credentials and taps "Login"
Description	Authenticate the user by email / phone / hospital code + password
Preconditions	PRE-1: Account exists
Postconditions	POST-1: Valid accessToken/refreshToken issued; POST-2: Navigation routes by role
Normal Flow	2.0.1 User enters credentials + password2.0.2 System determines credential type (email / phone / hospital code)2.0.3 System compares password (bcrypt.compare)2.0.4 System generates accessToken (1h) + refreshToken (7d) with tokenVersion2.0.5 System navigates by role
Alternative Flows	2.1 Login by hospital code (BV_XXX) — System looks up Hospital.code then finds the hospital_admin User
Exceptions	2.0.E1 Account not found → 400 "Invalid credentials"2.0.E2 Account locked → 403 (BR-02)2.0.E3 Wrong password → 4002.0.E4 Hospital subscription expired → 403 (BR-04)
Priority	Must Have
Frequency of Use	Very high, every session
Business Rules	BR-01, BR-02, BR-04
Other Information	JWT contains userId, role, tokenVersion, hospitalId. Secret = JWT_SECRET || 'access_secret'
Assumptions	—

UC-03 — SSO Login (Google)

Assigned member: Nguyen Tuan Thanh

UC ID & Name	UC-03 — SSO Login (Google)
Assigned to	Nguyen Tuan Thanh
Primary Actor	Patient
Secondary Actors	Firebase Auth
Trigger	Patient taps "Login with Google"
Description	Sign in using a Google account verified through Firebase
Preconditions	PRE-1: Patient has a valid Google account
Postconditions	POST-1: User created or found matching the Google email; POST-2: accessToken issued
Normal Flow	3.0.1 Patient chooses Google login3.0.2 Firebase verifies and returns idToken3.0.3 System verifies idToken via Google API (identitytoolkit.googleapis.com)3.0.4 System finds or creates a User (role='patient')3.0.5 System issues tokens and navigates to Home
Alternative Flows	—
Exceptions	3.0.E1 Patient cancels the popup → return to Login3.0.E2 idToken invalid → 401
Priority	Should Have
Frequency of Use	Medium
Business Rules	—
Other Information	Mock mode supported: idToken='mock_google_token_123' for development
Assumptions	Web only (Firebase signInWithPopup)

UC-04 — Forgot Password

Assigned member: Nguyen Van Hoang

UC ID & Name	UC-04 — Forgot Password
Assigned to	Nguyen Van Hoang
Primary Actor	User (any role)

Secondary Actors	EmailService (Nodemailer)
Trigger	User taps "Forgot password"
Description	Recover password via an OTP code sent by email
Preconditions	PRE-1: Email is registered in the system
Postconditions	POST-1: New password saved (hashed); POST-2: tokenVersion bumped (invalidates old tokens)
Normal Flow	4.0.1 User enters email4.0.2 System generates a 6-digit OTP, 5-min TTL, saved on the User4.0.3 System emails the OTP via Nodemailer4.0.4 User enters OTP + new password4.0.5 System verifies OTP + expiry, hashes the new password, bumps tokenVersion
Alternative Flows	—
Exceptions	4.0.E1 OTP invalid or expired → 400 "Invalid OTP"
Priority	Must Have
Frequency of Use	Low
Business Rules	BR-01
Other Information	OTP stored as User.otpCode + User.otpExpires (5-min TTL)
Assumptions	User still has access to the registered email inbox

UC-05 — Activate Account (first login)

Assigned member: Nguyen Van Hoang

UC ID & Name	UC-05 — Activate Account (first login)
Assigned to	Nguyen Van Hoang
Primary Actor	Staff (Doctor/Nurse/Technician/Hospital Admin)
Secondary Actors	none
Trigger	First login with a temporary account created by hospital_admin / system admin
Description	Staff replaces the temporary password with a permanent one
Preconditions	PRE-1: isVerified=false on the account

Postconditions	POST-1: isVerified=true; POST-2: New password in effect; POST-3: tokenVersion bumped
Normal Flow	5.0.1 Staff logs in with the temporary account5.0.2 System detects requiresActivation=true and navigates to the ActivateAccount screen5.0.3 Staff enters currentPassword, newPassword, optional newEmail5.0.4 System PUT /auth/password — verifies current, hashes new, sets isVerified=true, bumps tokenVersion5.0.5 System navigates to the role-based Dashboard
Alternative Flows	—
Exceptions	5.0.E1 Wrong current password → 400 "Wrong current password"5.0.E2 newEmail already used → 409
Priority	Must Have
Frequency of Use	Once per new staff
Business Rules	BR-01
Other Information	Temp email pattern: @temp.neuroscan.internal — must be replaced during activation
Assumptions	—

UC-06 — View Patient Dashboard

Assigned member: Nguyen Tuan Thanh

UC ID & Name	UC-06 — View Patient Dashboard
Assigned to	Nguyen Tuan Thanh
Primary Actor	Patient
Secondary Actors	Reminder API
Trigger	Patient opens the app / enters the Overview tab
Description	View personal overview including today's medication reminder schedule (fetched from /api/v1/patient/reminders/today)
Preconditions	PRE-1: Logged in
Postconditions	POST-1: No data changes (read-only)

Normal Flow	6.0.1 System GET /auth/me retrieves the user info6.0.2 System GET /api/v1/patient/reminders/today retrieves today's reminders6.0.3 UI displays the "Schedule" widget with a mark-as-taken button (UC-10)
Alternative Flows	6.1 No reminders today ⟶ show "No medication scheduled today"
Exceptions	6.0.E1 Reminder API fails ⟶ widget hidden, the rest of the dashboard still renders
Priority	Must Have
Frequency of Use	Very high, every app open
Business Rules	BR-06 (Reminder Time Slots)
Other Information	Reminders fetched from the MedicineReminder collection, sorted by time
Assumptions	—

UC-07 — Declare Medical Record

Assigned member: Nguyen Tuan Thanh

UC ID & Name	UC-07 — Declare Medical Record
Assigned to	Nguyen Tuan Thanh
Primary Actor	Patient
Secondary Actors	AsyncStorage (local)
Trigger	Patient selects a document type in the Record Vault
Description	Self-declare and store personal medical record documents
Preconditions	PRE-1: Logged in
Postconditions	POST-1: Document saved (AsyncStorage locally + synced to server if patientId is linked)
Normal Flow	7.0.1 Patient selects document type (Lab Result, CT-Scan, MRI, etc.)7.0.2 Patient fills in DOC_SCHEMAS7.0.3 System saves to AsyncStorage + POST /api/v1/patient/records if online7.0.4 Document appears in the records list
Alternative Flows	—

Exceptions	—
Priority	Should Have
Frequency of Use	Medium
Business Rules	—
Other Information	Uses the useMedicalRecordForm hook — auto-saves draft to AsyncStorage
Assumptions	Data can be cached locally when offline and synced when online

UC-08 — View Visit History and MRI/CT Scans

Assigned member: Nguyen Tuan Thanh

UC ID & Name	UC-08 — View Visit History and MRI/CT Scans
Assigned to	Nguyen Tuan Thanh
Primary Actor	Patient
Secondary Actors	none
Trigger	Patient opens "MRI & CT Scans"
Description	Browse the list of past scans and related results
Preconditions	PRE-1: Logged in
Postconditions	— (read-only)
Normal Flow	8.0.1 System GET /api/v1/imaging/my-results by patientId8.0.2 UI displays a timeline with MRI/CT thumbnails8.0.3 Patient selects a result to view details (UC-09)
Alternative Flows	—
Exceptions	8.0.E1 No scans yet → show an empty list
Priority	Must Have
Frequency of Use	High
Business Rules	—
Other Information	Endpoint scoped by hospitalId via the tenancy plugin
Assumptions	—

UC-09 — View AI Analysis Result

Assigned member: Le Tran Gia Huy

UC ID & Name	UC-09 — View AI Analysis Result
Assigned to	Le Tran Gia Huy
Primary Actor	Patient
Secondary Actors	none
Trigger	Patient selects a scan result to view the AI analysis
Description	View the detailed AI analysis of an MRI/CT scan — including class (glioma / meningioma / pituitary / notumor), confidence, annotated image with bounding box, and Grad-CAM heatmap
Preconditions	PRE-1: AI result is available for this scan
Postconditions	— (read-only)
Normal Flow	9.0.1 System GET /api/v1/imaging/:id retrieves details9.0.2 UI displays the image + bounding box (YOLO / Gemini / OpenCV) + confidence + conclusion9.0.3 UI displays the doctor's confirmation status (approved / feedback)
Alternative Flows	—
Exceptions	—
Priority	Must Have
Frequency of Use	High
Business Rules	—
Other Information	Result includes: class_name, confidence, annotated_image (base64 JPEG), tumor_location, is_conflict, consensus_message, all_probabilities
Assumptions	AI result always requires doctor confirmation; it does not replace the official diagnosis

UC-10 — Mark Medication as Taken

Assigned member: Nguyen Tuan Thanh

UC ID & Name	UC-10 — Mark Medication as Taken
Assigned to	Nguyen Tuan Thanh
Primary Actor	Patient
Secondary Actors	none
Trigger	Patient taps "Mark as taken" on a reminder
Description	Mark a daily medication reminder as completed
Preconditions	PRE-1: Reminder exists and belongs to this patient
Postconditions	POST-1: status='done' on that reminder
Normal Flow	10.0.1 Patient taps the mark-as-taken button10.0.2 System PUT /api/v1/patient/reminders/:id/done10.0.3 System updates MedicineReminder.status='done'10.0.4 System refreshes the reminder list
Alternative Flows	—
Exceptions	10.0.E1 Reminder not found ⟶ 404
Priority	Must Have
Frequency of Use	High (multiple times per day)
Business Rules	BR-06 (Reminder Time Slots)
Other Information	MedicineReminder model — index {patientId, date, status}
Assumptions	—

UC-11 — Upgrade to Premium Plan

Assigned member: Nguyen Van Hoang

UC ID & Name	UC-11 — Upgrade to Premium Plan
Assigned to	Nguyen Van Hoang
Primary Actor	Patient
Secondary Actors	VNPay/MoMo, Subscription model

Trigger	Patient taps "Upgrade now" on the PremiumScreen
Description	Pay to upgrade the service plan via VNPay or MoMo
Preconditions	PRE-1: Patient does not already have an active Premium plan
Postconditions	POST-1: Subscription.status='active' after successful payment; POST-2: expiresAt = now + 365 days
Normal Flow	11.0.1 Patient chooses to upgrade11.0.2 System POST /billing/b2c/checkout {provider:'vnpay'} creates a pending Subscription + paymentUrl11.0.3 Patient completes payment on the VNPay/MoMo gateway11.0.4 Gateway calls POST /billing/webhook/:provider with txnRef + status11.0.5 System updates Subscription.status='active', startedAt, expiresAt11.0.6 FE polls GET /billing/b2c/status — shows "Upgrade successful"
Alternative Flows	—
Exceptions	11.0.E1 Payment failed / cancelled → Subscription.status='expired', current plan unchanged
Priority	Must Have (B2C revenue)
Frequency of Use	Low
Business Rules	—
Other Information	Premium price: 99,000 VND/year. Providers: vnpay, momo. Webhook verifies provider signature (HMAC-SHA512 in production)
Assumptions	—

UC-12 — Create Visit

Assigned member: Nguyen Chau Quang

UC ID & Name	UC-12 — Create Visit
Assigned to	Nguyen Chau Quang
Primary Actor	Nurse
Secondary Actors	Notification Service
Trigger	Nurse opens the "Create Visit" tab and selects a patient
Description	Create a new visit, assigning patient — doctor — nurse

Preconditions	PRE-1: Patient, doctor, nurse all belong to the same hospital as the Nurse (BR-05)PRE-2: Today's visit count < pricing.maxPatients
Postconditions	POST-1: New Visit with status='waiting'; POST-2: Notification (type='new_visit') created for doctor + nurse
Normal Flow	12.0.1 Nurse GET /api/v1/visits/staff retrieves doctors + nurses + technicians12.0.2 Nurse searches & selects patient, doctor, nurse, reason12.0.3 FE POST /api/v1/visits {patientId, doctorId, nurseId, reason, visitType}12.0.4 BE protect middleware verifies JWT + tenantStorage.run({hospitalId})12.0.5 BE checks the maxPatients pricing limit12.0.6 BE creates Visit with hospitalId auto-injected by the tenancy plugin12.0.7 BE createNotificationInternal for doctor + nurse12.0.8 FE switches to the "Today's Visits" tab
Alternative Flows	—
Exceptions	12.0.E1 Missing required field → 40012.0.E2 Visit count >= maxPatients → 400 "Daily patient limit reached"12.0.E3 Doctor / nurse in a different hospital → 403 (tenancy plugin auto-filter)
Priority	Must Have
Frequency of Use	Very high, every patient arrival
Business Rules	BR-05 (Cross-Hospital Referral)
Other Information	Notification created via createNotificationInternal — never throws (logs on failure)
Assumptions	—

UC-13 — View Work Queue

Assigned member: Le Hai Nam

UC ID & Name	UC-13 — View Work Queue
Assigned to	Le Hai Nam
Primary Actor	Doctor
Secondary Actors	none
Trigger	Doctor opens "Work Queue"
Description	View the list of pending / in-progress visits assigned to the doctor

Preconditions	PRE-1: Logged in with role doctor
Postconditions	— (read-only)
Normal Flow	13.0.1 System GET /api/v1/visits/my-queue13.0.2 BE filters Visit by doctorId, status ∈ {waiting, in-progress, awaiting AI result, awaiting doctor review}13.0.3 UI displays the "In Progress" list13.0.4 Doctor picks a visit to start ⟶ navigation.navigate('PatientDetail', {patientId, visitId})
Alternative Flows	—
Exceptions	—
Priority	Must Have
Frequency of Use	Very high
Business Rules	—
Other Information	getMyQueue is role-aware: doctor sees own visits + MRI pipeline; technician sees assigned / unassigned MRI scans
Assumptions	—

UC-14 — Order MRI Scan

Assigned member: Le Hai Nam

UC ID & Name	UC-14 — Order MRI Scan
Assigned to	Le Hai Nam
Primary Actor	Doctor
Secondary Actors	Notification Service
Trigger	Doctor taps "Order MRI" on a visit
Description	Issue an MRI scan order and assign a technician
Preconditions	PRE-1: Visit is currently in the examining state
Postconditions	POST-1: status='awaiting-scan'; POST-2: mriOrder populated; POST-3: Notification (type='mri_order') sent to the technician

Normal Flow	14.0.1 Doctor selects technician, scan region, instructions, requestAiAnalysis flag14.0.2 FE PUT /api/v1/visits/:id/mri-order14.0.3 BE updates Visit with mriOrder + status='awaiting-scan'14.0.4 BE createNotificationInternal for the technician
Alternative Flows	—
Exceptions	—
Priority	Must Have
Frequency of Use	High
Business Rules	BR-05 (Cross-Hospital Referral)
Other Information	mriOrder sub-doc: {region, instructions, requestAiAnalysis, imagingResultId, orderedAt}
Assumptions	—

UC-15 — Process MRI / PACS Queue

Assigned member: Le Tran Gia Huy

UC ID & Name	UC-15 — Process MRI / PACS Queue
Assigned to	Le Tran Gia Huy
Primary Actor	Technician
Secondary Actors	Firebase Storage (image upload)
Trigger	Technician opens the "MRI Scan Room"
Description	Update the status of the scanning process and upload MRI/CT images
Preconditions	PRE-1: At least one Visit in the awaiting-scan / scanning state
Postconditions	POST-1: Visit status updated (scanning / awaiting AI result); POST-2: ImagingResult created with images[]

Normal Flow	15.0.1 Technician filters the list by status='awaiting-scan'15.0.2 Technician taps "Start scan" → PUT /api/v1/visits/:id/status {status:'scanning'}15.0.3 Technician uploads base64 images → POST /api/v1/imaging/upload15.0.4 BE uploads to Firebase Storage, returns a public URL15.0.5 Technician creates an ImagingResult → POST /api/v1/imaging-results15.0.6 BE notifies the doctor (type='ai_ready') + updates Visit status='awaiting AI result'
Alternative Flows	—
Exceptions	15.0.E1 Firebase upload fails → fallback to local /uploads/
Priority	Must Have
Frequency of Use	High
Business Rules	BR-05
Other Information	createKtvImagingResult auto-fills placeholder fields from patient / visit
Assumptions	—

UC-16 — Prescribe Medication (Auto-generate Reminders)

Assigned member: Le Hai Nam

UC ID & Name	UC-16 — Prescribe Medication (Auto-generate Reminders)
Assigned to	Le Hai Nam
Primary Actor	Doctor
Secondary Actors	MedicineReminder (auto-generated)
Trigger	Doctor taps "Examine & Prescribe" from the work queue
Description	Prescribe medication; the system auto-generates the medication reminder schedule per BR-06
Preconditions	PRE-1: Patient is within the doctor's access scope (same hospital or valid transfer)
Postconditions	POST-1: New Prescription saved; POST-2: MedicineReminder auto-generated for each drug × each day × each time slot

Normal Flow	16.0.1 Doctor enters the diagnosis16.0.2 Doctor adds drugs (quantity, usage, timesPerDay, durationDays)16.0.3 FE POST /api/patients/:patientId/prescriptions {diagnosis, drugs[], note}16.0.4 BE checkPatientTenancy — verifies same hospital16.0.5 BE creates a new Prescription16.0.6 BE generateRemindersForPrescription: loops for each drug × day × time slot (BR-06) → MedicineReminder.insertMany16.0.7 UI displays the newly created prescription
Alternative Flows	—
Exceptions	16.0.E1 Missing diagnosis or empty drugs list → 40016.0.E2 generateReminders fails — does not block the main flow; logs the error
Priority	Must Have
Frequency of Use	High
Business Rules	BR-05, BR-06 (Reminder Time Slots: 1→08:00; 2→08:00,20:00; 3→+13:00; 4→+12:00,17:00,21:00)
Other Information	REMINDER_TIME_SLOTS map in reminder.controller.js
Assumptions	—

UC-17 — End Visit / Transfer Patient

Assigned member: Le Hai Nam

UC ID & Name	UC-17 — End Visit / Transfer Patient
Assigned to	Le Hai Nam
Primary Actor	Doctor
Secondary Actors	Invoice (auto-created)
Trigger	Doctor taps "End visit"
Description	Close the visit, generate an invoice, or transfer the patient to another hospital
Preconditions	PRE-1: Visit is currently in the examining state
Postconditions	POST-1a (complete): status='completed' + Invoice auto-created (exam + MRI + AI + drug items); POST-1b (transfer): status='closed' + TransferForm records the destination hospital

Normal Flow	17.0.1 Doctor chooses the treatment direction (Complete / Transfer)17.0.2 FE PUT /api/v1/visits/:id/status {status:'completed'} or {status:'closed', transferTo}17.0.3 BE updates the Visit status17.0.4 (Complete only) BE auto-creates Invoice: items = [exam + MRI (if mriOrder.orderedAt) + AI (if requestAiAnalysis) + drug items from Prescription]17.0.5 BE updates the queue
Alternative Flows	17.1 Complete visit → Invoice auto-created with drug items pulled from the Prescription17.2 Transfer → TransferForm created with the transferTo field
Exceptions	—
Priority	Must Have
Frequency of Use	High
Business Rules	—
Other Information	Drug items priced via Drug.price; prescription marked isBilled when the invoice is paid (UC-18)
Assumptions	—

UC-18 — Confirm Invoice Payment

Assigned member: Nguyen Van Hoang

UC ID & Name	UC-18 — Confirm Invoice Payment
Assigned to	Nguyen Van Hoang
Primary Actor	Nurse / Cashier
Secondary Actors	AuditLog, EMRVersion
Trigger	Nurse opens the "Billing" tab and selects a pending invoice
Description	Confirm cash / transfer payment for a visit invoice
Preconditions	PRE-1: Invoice is in the unpaid state
Postconditions	POST-1: status='paid', paidAt=now; POST-2: Drug.stock decremented; POST-3: Prescription.isBilled=true; POST-4: Visit.status='closed'; POST-5: EMRVersion audit entry created

Normal Flow	18.0.1 Nurse GET /api/v1/invoices retrieves unpaid invoices18.0.2 Nurse selects an invoice, taps "Confirm payment" + paymentMethod18.0.3 FE PUT /api/v1/invoices/:id/pay {paymentMethod:'cash'|'transfer'}18.0.4 BE payInvoice: re-aggregates drug items, decrements Drug.stock, marks Prescription.isBilled, sets Visit.status='closed'18.0.5 BE creates an EMRVersion audit entry recording the payment18.0.6 BE returns 200
Alternative Flows	—
Exceptions	—
Priority	Must Have
Frequency of Use	High
Business Rules	—
Other Information	Invoice item types: exam, mri, ai, drug, other. Stock decrement: Drug.stock.quantity -= item.quantity
Assumptions	Cash at the counter or bank transfer

UC-19 — Assign Work Shifts

Assigned member: Nguyen Chau Quang

UC ID & Name	UC-19 — Assign Work Shifts
Assigned to	Nguyen Chau Quang
Primary Actor	Hospital Admin
Secondary Actors	none
Trigger	Admin selects a staff member + date and taps "+ Add shift"
Description	Assign on-call shifts to staff, supporting multiple shifts per day (morning / afternoon / evening / full-day)
Preconditions	PRE-1: Staff belongs to the same hospital as the Admin
Postconditions	POST-1: New WorkSchedule created

Normal Flow	19.0.1 Admin selects shift type (morning / afternoon / evening / full-day), start/end time19.0.2 FE findSchedules(staffId, date) — checks existing shifts that day19.0.3 FE POST /api/v1/schedules {staffId, date, shift, startTime, endTime}19.0.4 BE createSchedule — checks duplicate shift type (BR-03) via unique index {staffId, date, shift}19.0.5 BE creates the new WorkSchedule
Alternative Flows	—
Exceptions	19.0.E1 This exact shift already exists that day → 409 (BR-03, unique index violation)
Priority	Must Have
Frequency of Use	High (weekly)
Business Rules	BR-03 (Duplicate Shift Prevention)
Other Information	Compound indexes: {hospitalId, date} and unique {staffId, date, shift}
Assumptions	—

UC-20 — Request Shift Swap

Assigned member: Nguyen Chau Quang

UC ID & Name	UC-20 — Request Shift Swap
Assigned to	Nguyen Chau Quang
Primary Actor	Staff (Doctor / Nurse / Technician)
Secondary Actors	none
Trigger	Staff taps "Request shift swap" on a specific shift
Description	Staff requests to swap an on-call shift to another day / person
Preconditions	PRE-1: The shift belongs to the staff member
Postconditions	POST-1: New SwapRequest with status='pending'
Normal Flow	20.0.1 Staff selects the target date, swap partner (optional), reason20.0.2 FE POST /api/v1/schedules/swap-requests {scheduleId, targetDate, targetStaffId, reason}20.0.3 BE createSwapRequest creates a SwapRequest with status='pending'20.0.4 UI shows "Pending approval" status

Alternative Flows	—
Exceptions	—
Priority	Should Have
Frequency of Use	Medium
Business Rules	—
Other Information	SwapRequest fields: requesterId, scheduleId, targetStaffId, targetDate, reason, status, reviewedBy, reviewNotes
Assumptions	—

UC-21 — Review Swap Request

Assigned member: Nguyen Chau Quang

UC ID & Name	UC-21 — Review Swap Request
Assigned to	Nguyen Chau Quang
Primary Actor	Hospital Admin
Secondary Actors	EmailService
Trigger	Admin opens the "Swap Requests" tab and chooses approve / reject
Description	Approve or reject a staff member's shift swap request
Preconditions	PRE-1: SwapRequest is in the pending state
Postconditions	POST-1a (approve): WorkSchedule swapped + SwapRequest.status='approved'; POST-1b (reject): status='rejected'; POST-2: Notification email sent
Normal Flow	21.0.1 Admin views the "Pending" list21.0.2 Admin chooses Approve or Reject + reviewNotes21.0.3 FE PUT /api/v1/schedules/swap-requests/:id {status, reviewNotes}21.0.4 BE reviewSwapRequest: if approved + targetStaffId set ⟶ swap staffId between the two schedules; if approved + targetDate set ⟶ update schedule.date21.0.5 BE sendSwapRequestResultEmail via Nodemailer
Alternative Flows	—
Exceptions	—

Priority	Should Have
Frequency of Use	Medium
Business Rules	—
Other Information	Email template: sendSwapRequestResultEmail({toEmail, staffName, status, shiftDetails, reviewNotes})
Assumptions	—

UC-22 — Update Drug Stock

Assigned member: Le Van Minh

UC ID & Name	UC-22 — Update Drug Stock
Assigned to	Le Van Minh
Primary Actor	Nurse / Hospital Admin
Secondary Actors	Notification Service
Trigger	Selects a drug and enters incoming / outgoing quantity
Description	Stock in / out for a drug, with low-stock alerts
Preconditions	PRE-1: Drug exists in the catalog
Postconditions	POST-1: Stock updated; POST-2 (if below threshold): Admin receives a Notification type='low_stock'
Normal Flow	22.0.1 User selects an action (set | add | subtract) + quantity22.0.2 FE POST /api/drugs/:id/stock {action, quantity}22.0.3 BE updateStock: computes newStock, prevents negative22.0.4 BE checks isLowStock (stock.quantity < stock.minStock)22.0.5 (Low stock only) BE createNotificationInternal for hospital_admins22.0.6 UI shows the new stock + warning if any
Alternative Flows	—
Exceptions	22.0.E1 newStock < 0 ⟶ 400 "Invalid quantity"
Priority	Should Have
Frequency of Use	Medium
Business Rules	—
Other Information	Drug.stock: {quantity, unit, minStock, lastUpdated}

Assumptions	—

UC-23 — Provision Temp Hospital Account

Assigned member: Nguyen Van Hoang

UC ID & Name	UC-23 — Provision Temp Hospital Account
Assigned to	Nguyen Van Hoang
Primary Actor	System Admin
Secondary Actors	EmailService, Google Drive API
Trigger	Admin enters the hospital name + IT contact email
Description	Generate a hospital code + a temporary hospital_admin account + Google Drive folder structure
Preconditions	PRE-1: Hospital has not been previously provisioned
Postconditions	POST-1: New Hospital with status='provisioned'; POST-2: Temporary hospital_admin User (email @temp.neuroscan.internal) created; POST-3: Drive folder structure (5 subfolders) created; POST-4: Email with credentials sent
Normal Flow	23.0.1 Admin enters hospital name + IT email23.0.2 BE generates hospital code BV_XXX + temp password BV@XXXXXXXX23.0.3 BE setupHospitalDriveStructure — creates the main folder + 5 subfolders (Original_Scans, AI_Predictions, Doctor_Revisions, Patient_Reports, Metadata_Backups)23.0.4 BE creates Hospital with driveFolderId/Url + temporary hospital_admin User23.0.5 BE sendHospitalCredentials({itEmail, hospitalName, tempUsername, tempPassword})23.0.6 UI displays the BV code + password for handover
Alternative Flows	—
Exceptions	23.0.E1 Drive API fails — Hospital is still created but driveFolderId is empty (admin can retry)
Priority	Must Have
Frequency of Use	Low (per new hospital)
Business Rules	BR-07 (Onboarding Sequence)
Other Information	Temp email pattern: <code>@temp.neuroscan.internal — must be replaced in UC-25

Assumptions	—

UC-24 — Submit Onboarding Information

Assigned member: Le Van Minh

UC ID & Name	UC-24 — Submit Onboarding Information
Assigned to	Le Van Minh
Primary Actor	Hospital Admin
Secondary Actors	Google Cloud Storage (license upload)
Trigger	Logs in with the BV_XXX code (temp account) and fills the onboarding form
Description	Hospital submits full legal / contact information + uploads the business license
Preconditions	PRE-1: Hospital is in status='provisioned'
Postconditions	POST-1: status='submitted'; POST-2: License file uploaded to GCS
Normal Flow	24.0.1 Hospital Admin GET /api/v1/hospital/me — sees status='provisioned'24.0.2 FE navigates to HospitalOnboardingScreen (5 sections, 18 fields)24.0.3 HAdmin fills: name, nameShort, taxCode, loginEmail, address (street/ward/district/province), phone, contactEmail, website, fax, legalRep{name,position,phone,email}, itContact{name,phone,email}24.0.4 HAdmin uploads the business license via expo-document-picker → POST /api/v1/hospital/onboarding/license (FormData) → uploadToGCS24.0.5 FE PUT /api/v1/hospital/onboarding {...fields}24.0.6 BE validates: Vietnamese phone regex, email regex, taxCode numeric + unique24.0.7 BE updates Hospital.status='submitted'
Alternative Flows	—
Exceptions	24.0.E1 Missing required field → 400, cannot submit24.0.E2 taxCode duplicate → 40924.0.E3 loginEmail matches another user → 409
Priority	Must Have
Frequency of Use	Low (once per hospital)
Business Rules	BR-07

Other Information	Vietnamese phone regex: ^(0|\+84)(3|5|7|8|9|2)\d{8,9}$
Assumptions	—

UC-25 — Activate Hospital

Assigned member: Le Van Minh

UC ID & Name	UC-25 — Activate Hospital
Assigned to	Le Van Minh
Primary Actor	System Admin
Secondary Actors	AuditLog
Trigger	Admin views a hospital with status='submitted' and taps "Verify"
Description	Verify and officially activate the hospital account
Preconditions	PRE-1: Hospital.status='submitted' (BR-07 — depends on UC-24 being complete; this is a mandatory business workflow sequence, not an extend relationship)
Postconditions	POST-1: status='active'; POST-2: Temp email replaced with the official email on the hospital_admin User; POST-3: isVerified=true; POST-4: AuditLog recorded
Normal Flow	25.0.1 Admin views the Hospital detail with status='submitted'25.0.2 Admin taps "Verify account"25.0.3 FE PUT /admin/hospitals/:id/activate25.0.4 BE activateHospital: checks status='submitted'25.0.5 BE updates User: email = hospital.loginEmail (replaces @temp.neuroscan.internal), isVerified=true25.0.6 BE updates Hospital.status='active'25.0.7 BE AuditLog.create({action:'activate-hospital', performedBy: admin.id})
Alternative Flows	—
Exceptions	25.0.E1 Hospital not yet status='submitted' ⟶ 400 "Onboarding not complete" (BR-07)
Priority	Must Have
Frequency of Use	Low
Business Rules	BR-07

Other Information	—
Assumptions	—

UC-26 — Lock / Unlock User Account

Assigned member: Le Van Minh

UC ID & Name	UC-26 — Lock / Unlock User Account
Assigned to	Le Van Minh
Primary Actor	System Admin
Secondary Actors	AuditLog
Trigger	Admin finds a user and taps "Lock / Unlock account"
Description	Lock or unlock any user account
Preconditions	PRE-1: User exists
Postconditions	POST-1: isLocked toggled; POST-2: AuditLog recorded
Normal Flow	26.0.1 Admin GET /admin/users — finds the user26.0.2 Admin taps lock / unlock26.0.3 FE PUT /admin/users/:id/lock (or /unlock)26.0.4 BE lockUserById (idempotent — skips AuditLog if state unchanged)26.0.5 BE updates User.isLocked26.0.6 BE AuditLog.create({action:'lock-user'|'unlock-user', performedBy: admin.id})
Alternative Flows	—
Exceptions	—
Priority	Must Have
Frequency of Use	Low
Business Rules	BR-02 (Account Lockout)
Other Information	Idempotent: if isLocked does not change, no extra AuditLog is written
Assumptions	—

UC-27 — View System Statistics

Assigned member: Le Van Minh

UC ID & Name	UC-27 — View System Statistics
Assigned to	Le Van Minh
Primary Actor	System Admin
Secondary Actors	none
Trigger	Admin opens "Admin Overview"
Description	View system-wide KPIs: totalUsers, totalDoctors, totalAiScans, totalRevenue, monthlyScans, recentActivities
Preconditions	PRE-1: Logged in with role admin
Postconditions	— (read-only)
Normal Flow	27.0.1 FE AdminMetricsView GET /admin/stats27.0.2 BE getSystemStats: aggregate User.countDocuments({role:'patient'/'doctor'}), ImagingResult.countDocuments, Invoice.aggregate totalRevenue27.0.3 BE computes monthlyScans (last 12 months), recentActivities (from AuditLog)27.0.4 UI shows KPI cards + line chart + recent activity feed
Alternative Flows	—
Exceptions	—
Priority	Should Have
Frequency of Use	High
Business Rules	—
Other Information	Returns: {stats: {totalUsers, totalDoctors, pendingDoctors, totalAiScans, totalRevenue, userDistribution, monthlyScans, recentActivities}}
Assumptions	—

UC-28 — Verify Tenant Isolation

Assigned member: Le Van Minh

UC ID & Name	UC-28 — Verify Tenant Isolation

Assigned to	Le Van Minh
Primary Actor	System Admin
Secondary Actors	none
Trigger	Admin taps "Start scan and verify isolation"
Description	Check whether data leaks across hospitals — runs a dry-run query isolation check
Preconditions	PRE-1: Logged in with role admin
Postconditions	— (read-only, report only)
Normal Flow	28.0.1 FE AdminSaaSSuiteView GET /admin/tenants/verify-isolation28.0.2 BE verifyTenantIsolation: Hospital.find() retrieves all hospitals28.0.3 Loop for each hospital h: Visit.countDocuments({hospitalId:{$ne:h._id}, performedByHospital: h._id}) — simulated leak check28.0.4 BE status = (leakCount==0) ? 'isolated' : 'leaked'28.0.5 BE returns {allIsolated, verificationResults[]}28.0.6 FE displays the result per hospital (Isolated / Leaked N staff from other hospitals)
Alternative Flows	—
Exceptions	—
Priority	Could Have (periodic security check)
Frequency of Use	Low
Business Rules	BR-05 (Cross-Hospital Referral)
Other Information	Code uses a simulated field performedByHospital for dry-run; production would check actual Visit.doctorId.hospitalId vs Visit.hospitalId
Assumptions	—

II.5.3 Activity Diagrams

Five activity diagrams are provided, each covering a major business process that spans multiple use cases and actors. Activity diagrams are drawn with swimlanes to show which actor performs each step.

II.5.3.1 Activity Diagram 1 — Patient Authentication & Registration

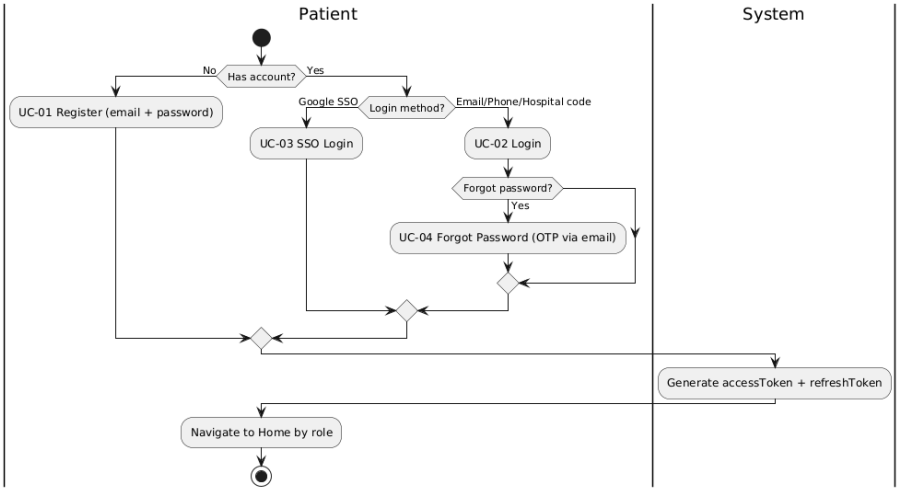

Figure 8: Activity Diagram — Authentication

Covers UC-01 (Register), UC-02 (Login), UC-03 (SSO), UC-04 (Forgot Password). The patient either registers a new account or logs in via email/phone/hospital code or Google SSO. On success, the system generates access + refresh tokens and navigates to the role-based Home screen.

II.5.3.2 Activity Diagram 2 — Clinical Visit Workflow

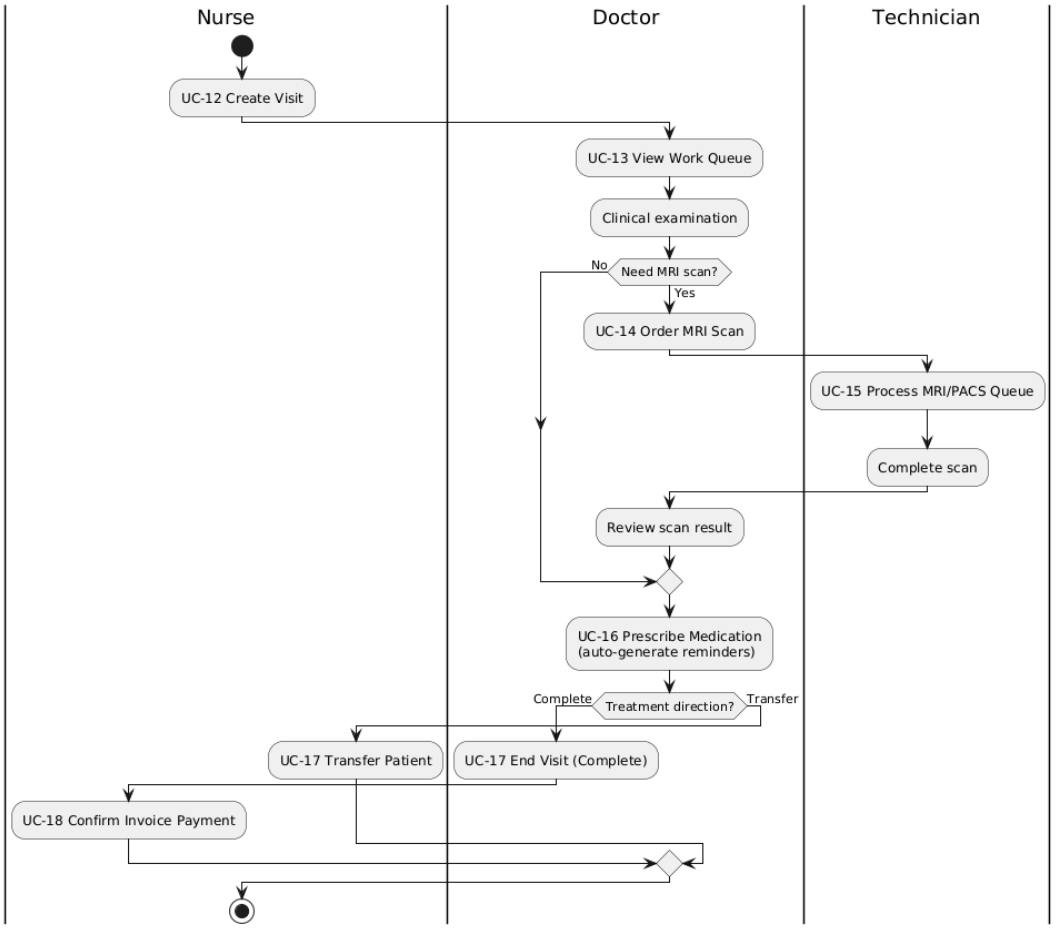

Figure 9: Activity Diagram — Clinical Visit

Covers UC-12 (Create Visit), UC-13 (View Work Queue), UC-14 (Order MRI), UC-15 (Process MRI Queue), UC-16 (Prescribe Medication), UC-17 (End Visit), UC-18 (Confirm Invoice Payment). Nurse creates the visit, doctor examines and optionally orders MRI, technician processes the scan, doctor prescribes medication (auto-generates reminders), then either completes the visit (auto-create invoice) or transfers the patient.

II.5.3.3 Activity Diagram 3 — Medication Prescription & Reminder Journey

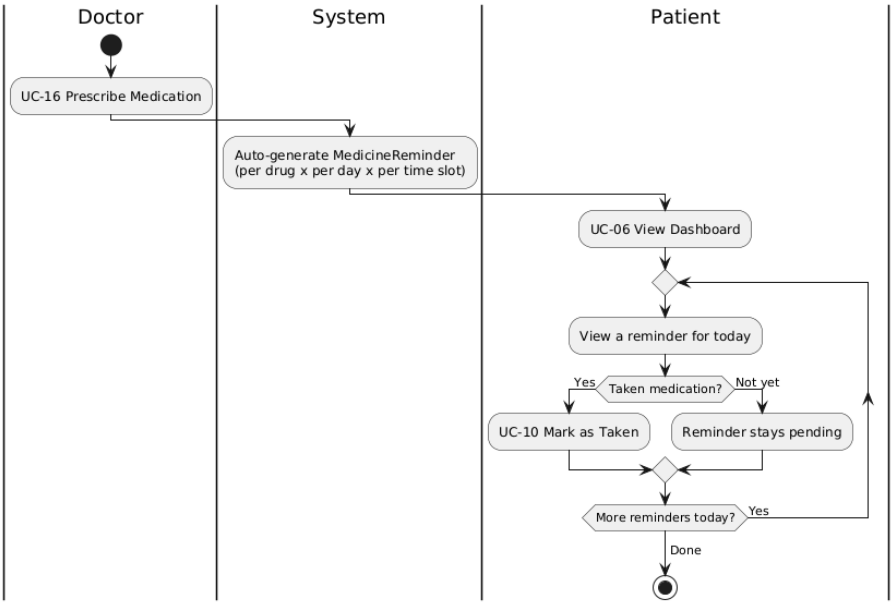

Figure 10: Activity Diagram — Medication

Covers UC-16 (Prescribe Medication) and UC-10 (Mark Medication as Taken). Doctor prescribes medication, system auto-generates MedicineReminder per drug × day × time slot (BR-06), patient views dashboard (UC-06) and marks each reminder as done throughout the day.

II.5.3.4 Activity Diagram 4 — Staff Scheduling & Swap Workflow

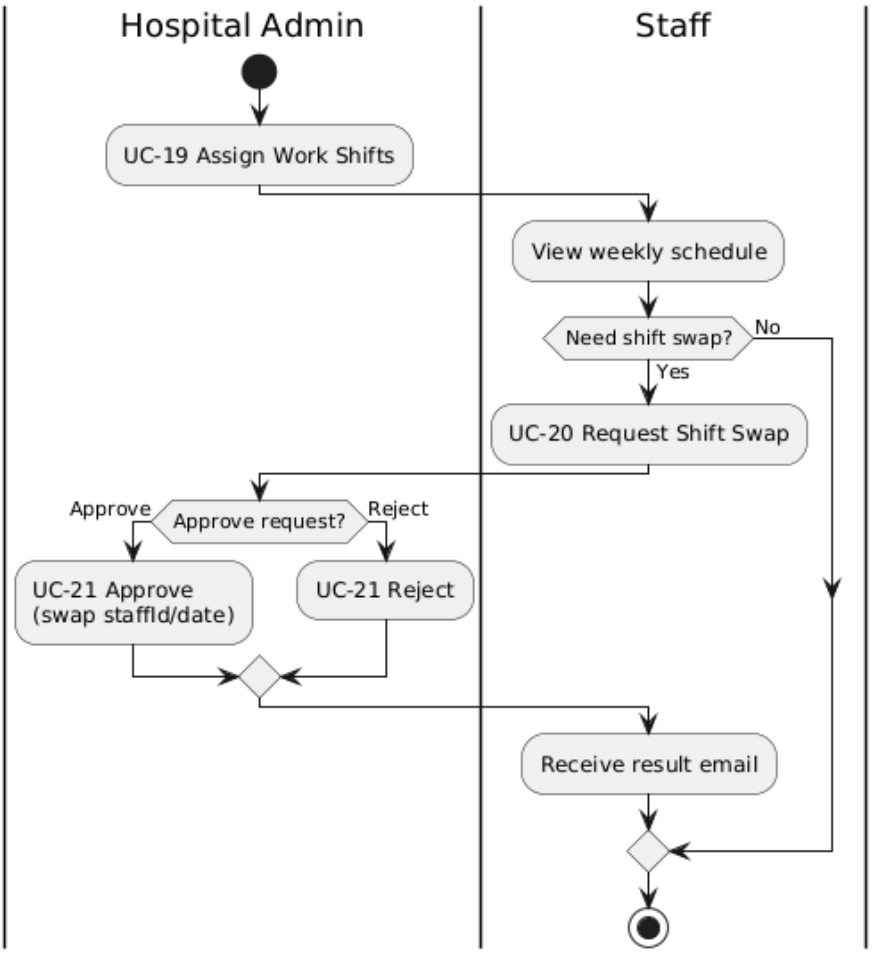

Figure 11: Activity Diagram — Scheduling

Covers UC-19 (Assign Work Shifts), UC-20 (Request Shift Swap), UC-21 (Review Swap Request). Hospital admin assigns shifts; staff view their weekly schedule and may request a swap; admin approves or rejects — email is sent to the requester with the result.

II.5.3.5 Activity Diagram 5 — Hospital Onboarding Workflow

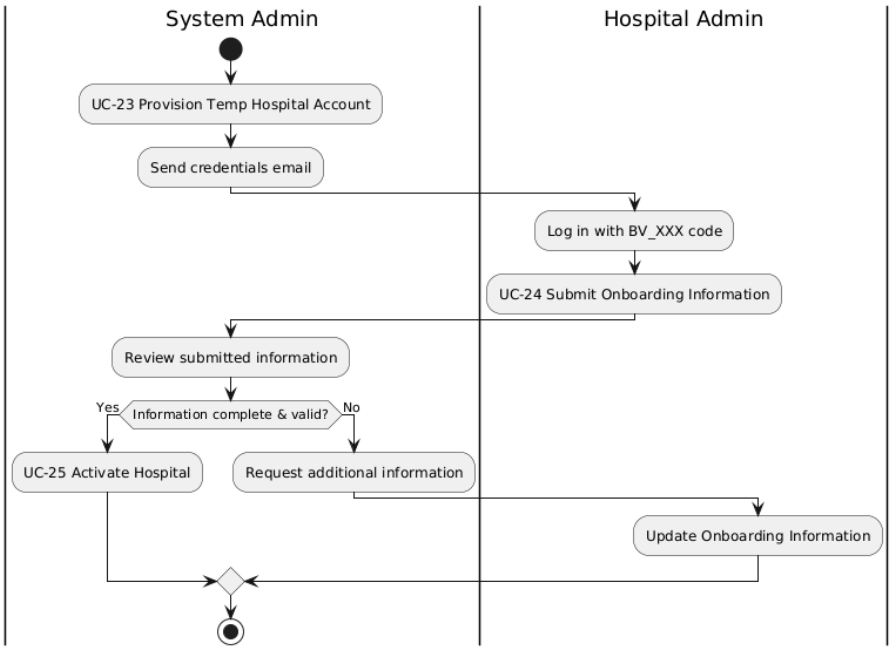

Figure 12: Activity Diagram — Onboarding

Covers UC-23 (Provision Temp Hospital Account), UC-24 (Submit Onboarding), UC-25 (Activate Hospital). System admin provisions a temp hospital account + Drive structure; hospital admin logs in with temp credentials, fills onboarding form, uploads license; system admin verifies and activates the hospital — changing the temp email to the official login email.

II.6 Entity Relationship Diagram

The ER diagram below shows the 14 core entities of NeuroScan AI (12 original + 2 newly added: MedicineReminder and Subscription) and their relationships. The full database schema is presented in Section IV.5.

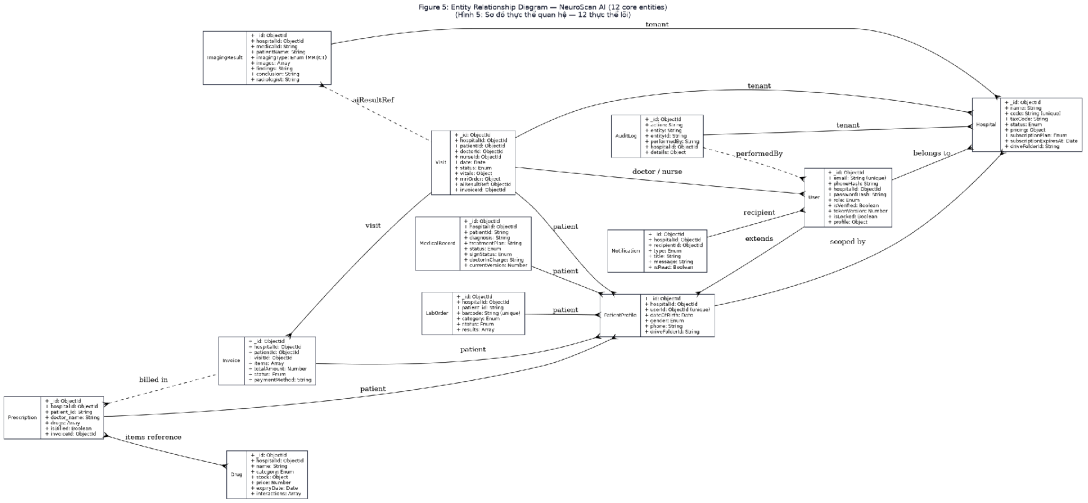

Figure 13: Entity Relationship Diagram

III. Analysis and Design Models

This section presents 28 sequence diagrams (one per use case), 2 state diagrams, and the supporting architectural diagrams. Each sequence diagram shows the exact API endpoints, controller methods, and database operations involved.

III.1 Sequence Diagrams

All 28 sequence diagrams are presented below in order UC-01 → UC-28. Each diagram shows the interactions between FE, API endpoints, controllers, and database collections.

III.1.1 UC-01 — Register Patient Account

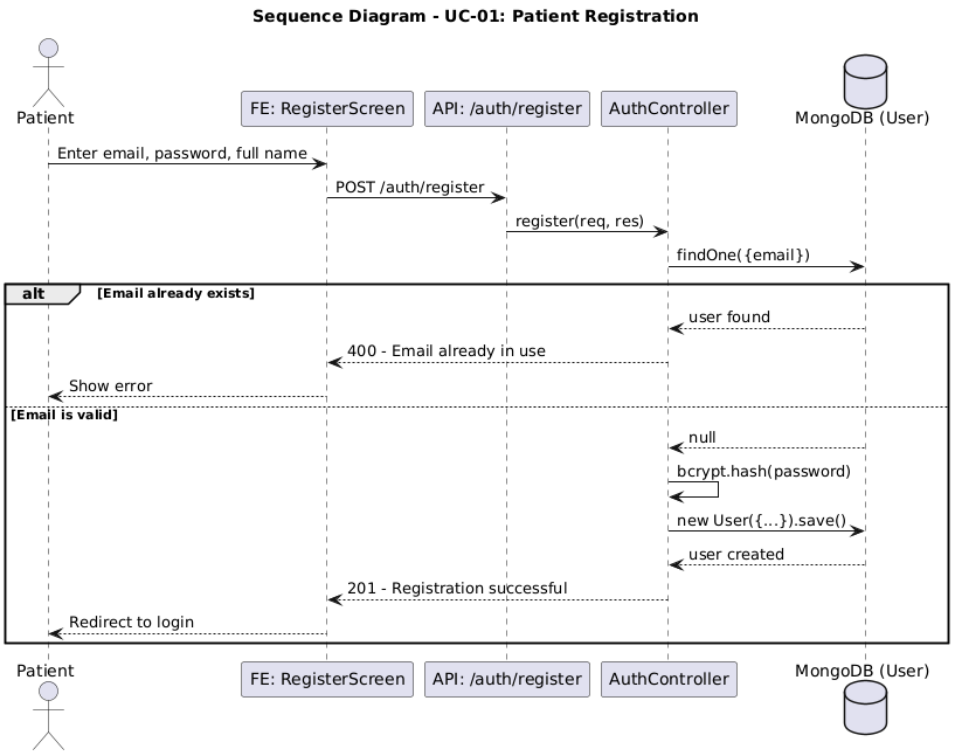

Figure 14: Sequence Diagram — UC-01

III.1.2 UC-02 — Login

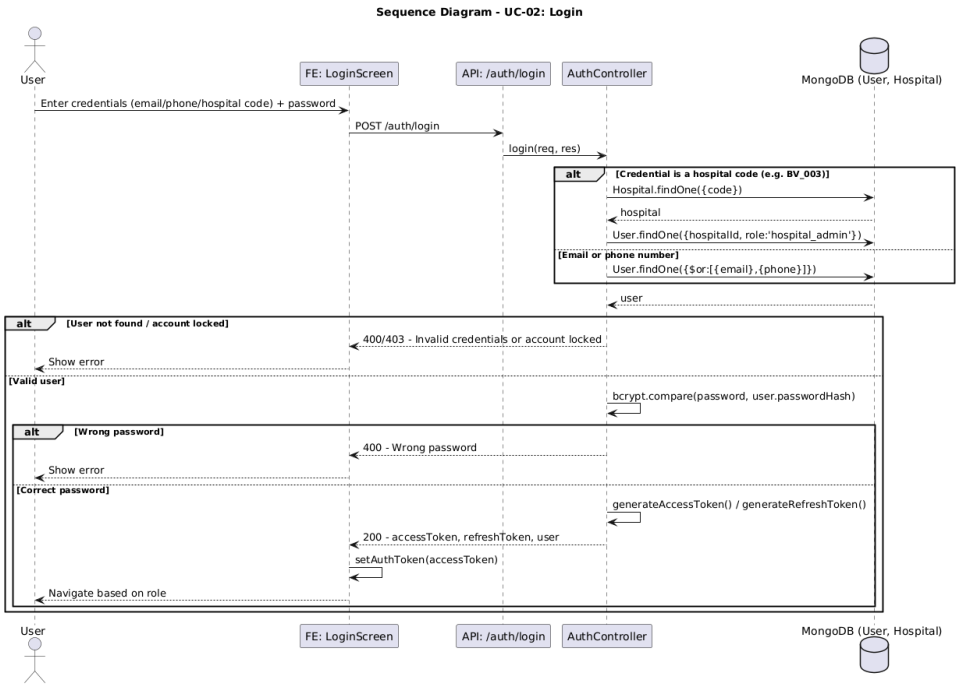

Figure 15: Sequence Diagram — UC-02

III.1.3 UC-03 — SSO Login (Google)

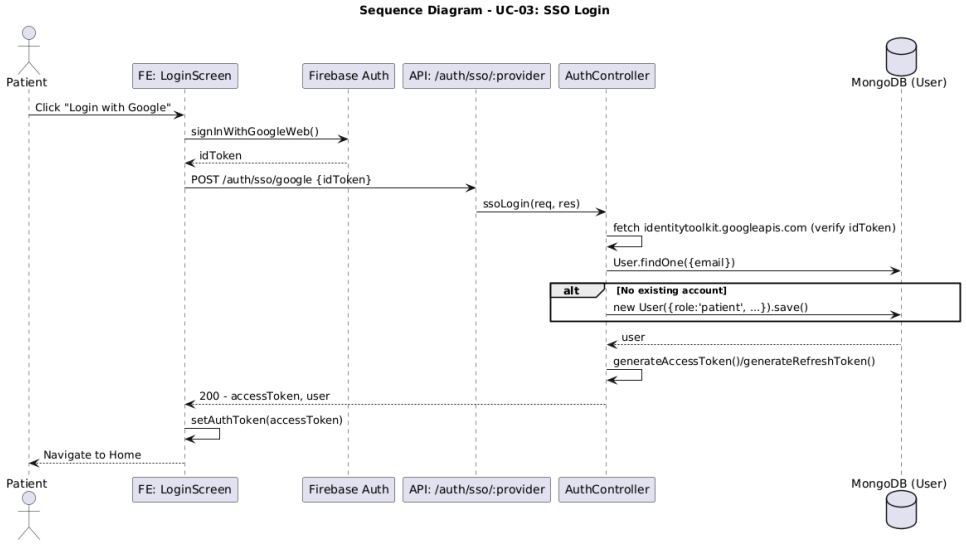

Figure 16: Sequence Diagram — UC-03

III.1.4 UC-04 — Forgot Password

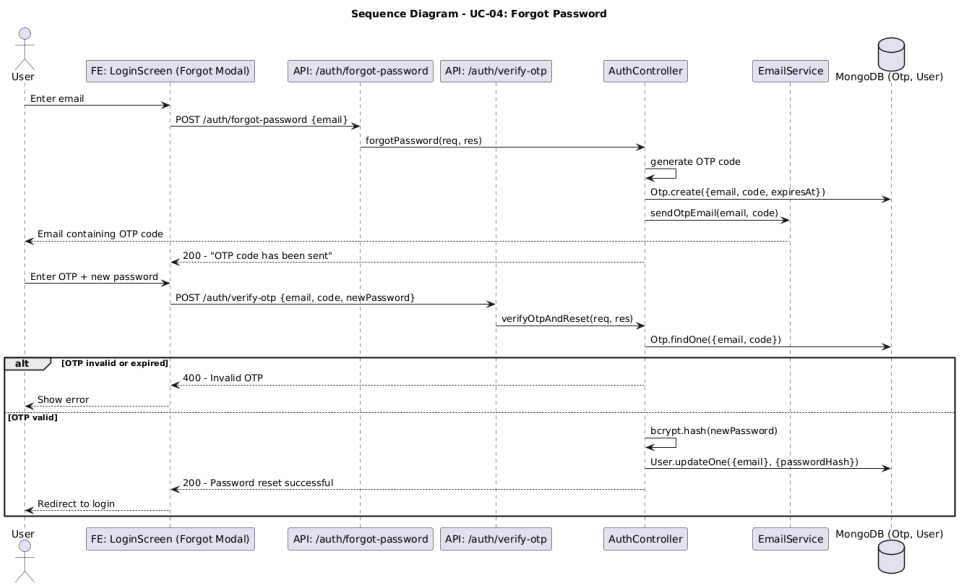

Figure 17: Sequence Diagram — UC-04

III.1.5 UC-05 — Activate Account (first login)

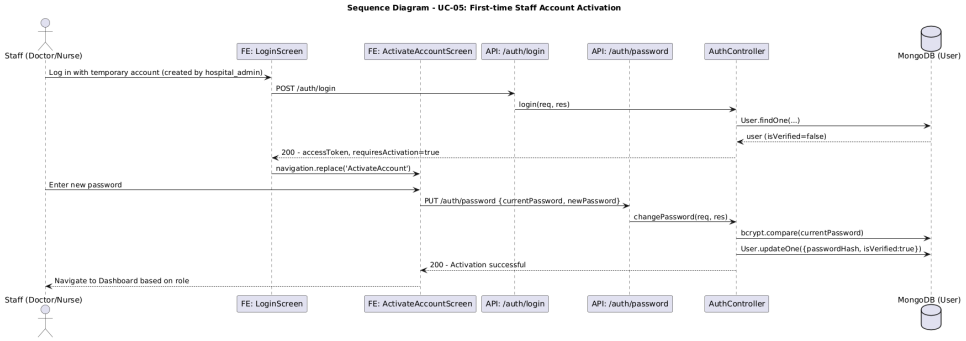

Figure 18: Sequence Diagram — UC-05

III.1.6 UC-06 — View Patient Dashboard

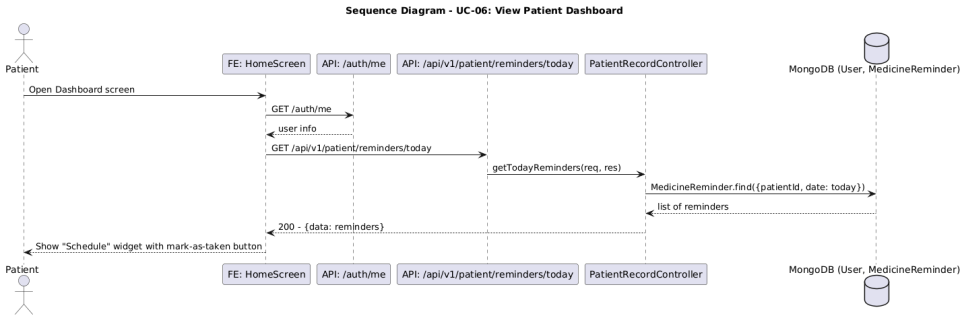

Figure 19: Sequence Diagram — UC-06

III.1.7 UC-07 — Declare Medical Record

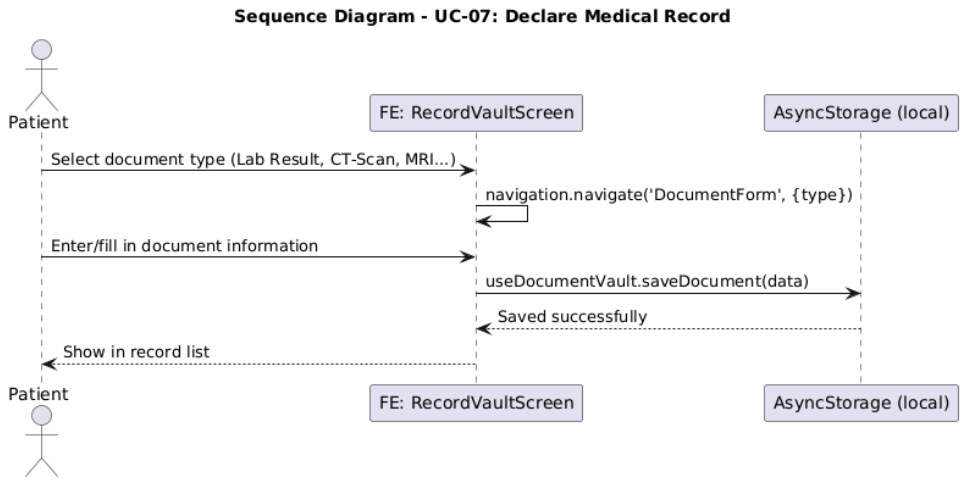

Figure 20: Sequence Diagram — UC-07

III.1.8 UC-08 — View Visit History and MRI/CT Scans

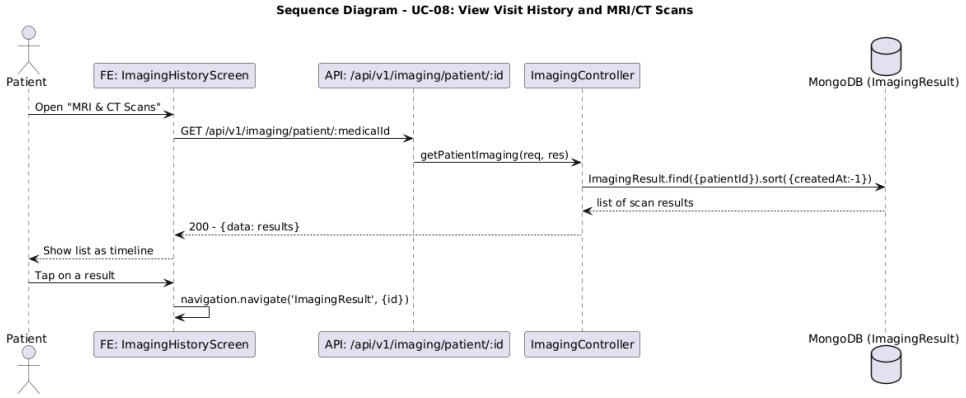

Figure 21: Sequence Diagram — UC-08

III.1.9 UC-09 — View AI Analysis Result

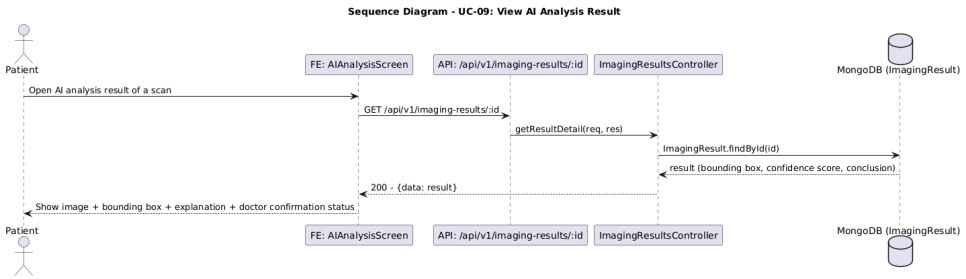

Figure 22: Sequence Diagram — UC-09

III.1.10 UC-10 — Mark Medication as Taken

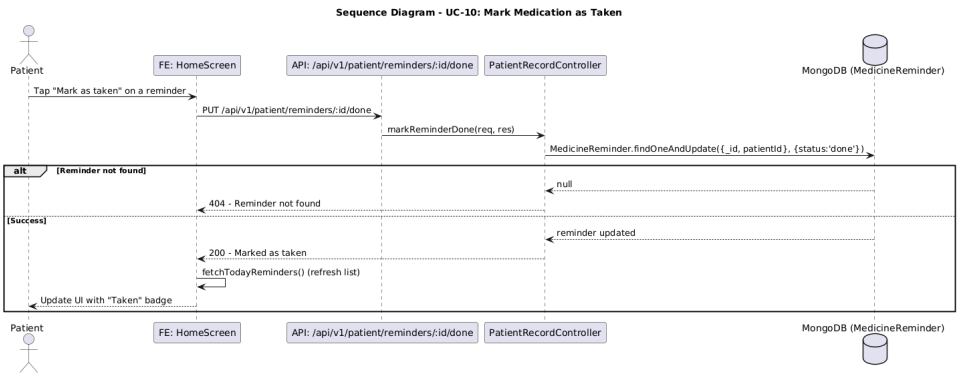

Figure 23: Sequence Diagram — UC-10

III.1.11 UC-11 — Upgrade to Premium Plan

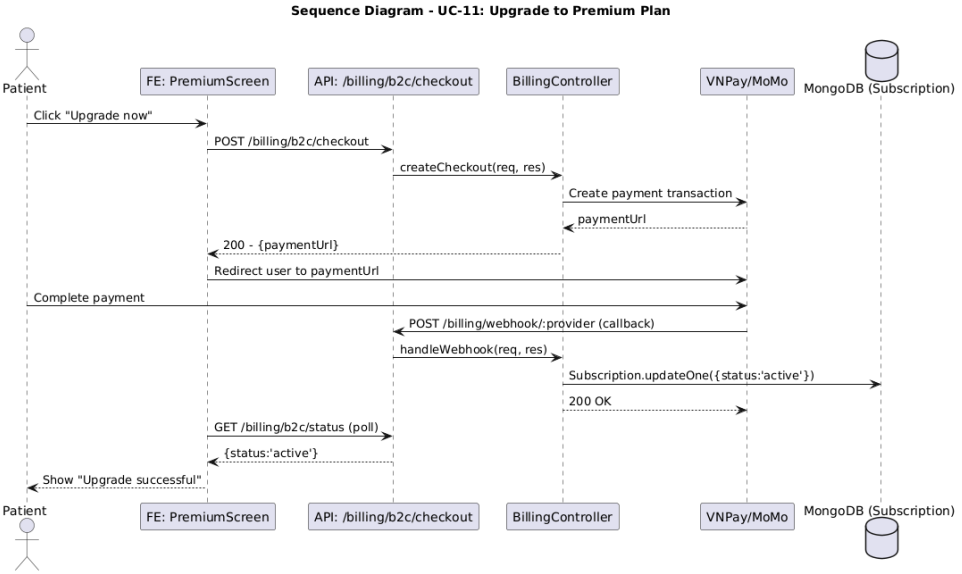

Figure 24: Sequence Diagram — UC-11

III.1.12 UC-12 — Create Visit

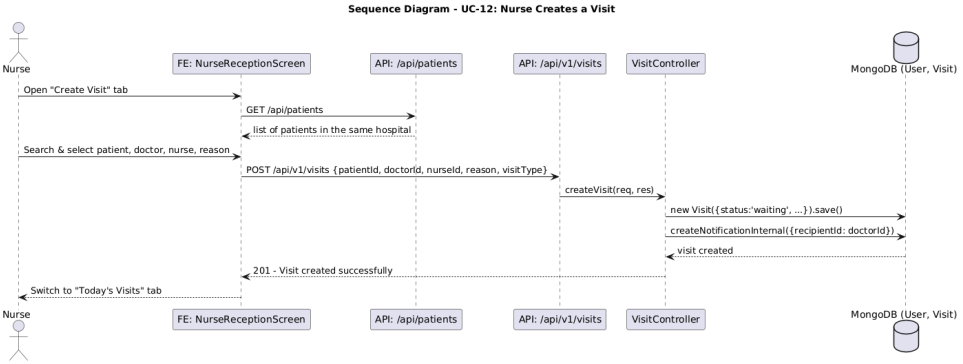

Figure 25: Sequence Diagram — UC-12

III.1.13 UC-13 — View Work Queue

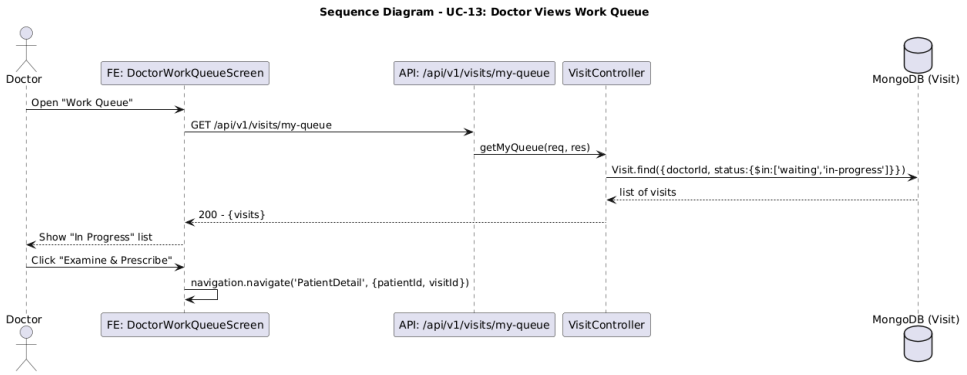

Figure 26: Sequence Diagram — UC-13

III.1.14 UC-14 — Order MRI Scan

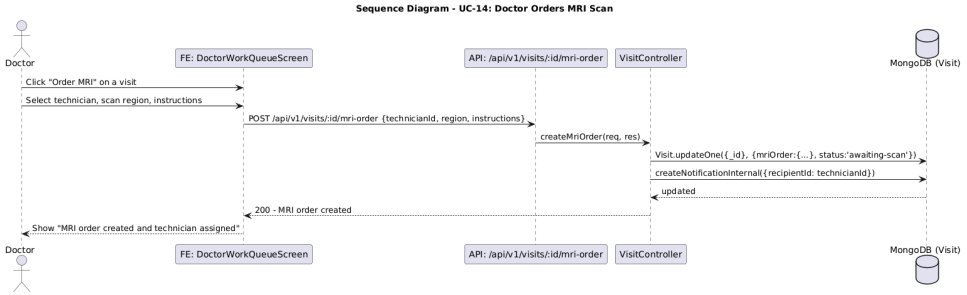

Figure 27: Sequence Diagram — UC-14

III.1.15 UC-15 — Process MRI / PACS Queue

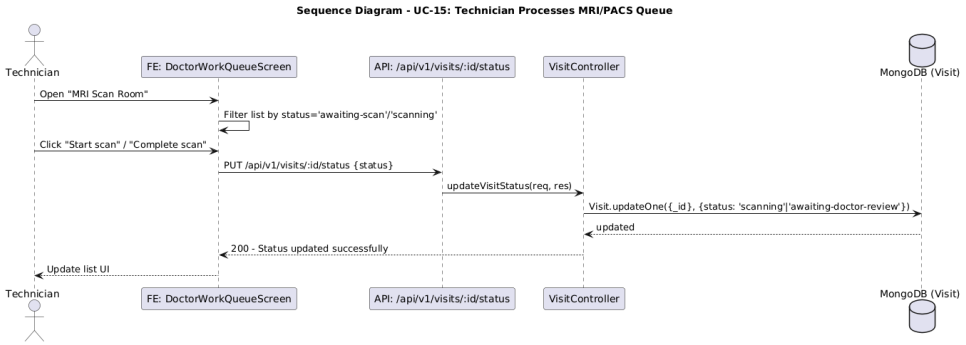

Figure 28: Sequence Diagram — UC-15

III.1.16 UC-16 — Prescribe Medication (Auto-generate Reminders)

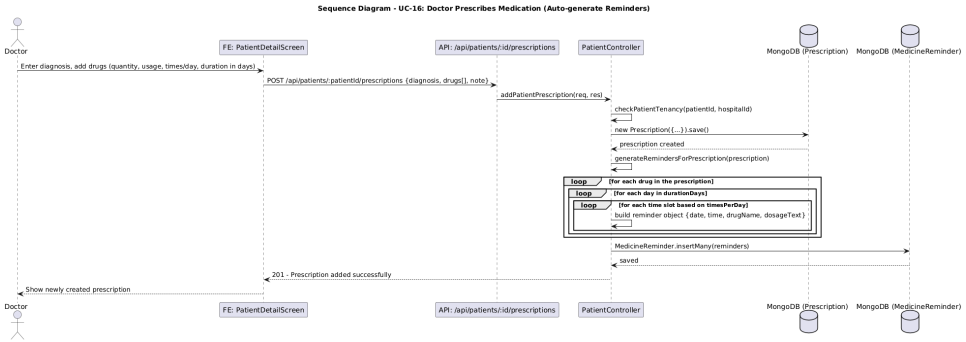

Figure 29: Sequence Diagram — UC-16

III.1.17 UC-17 — End Visit / Transfer Patient

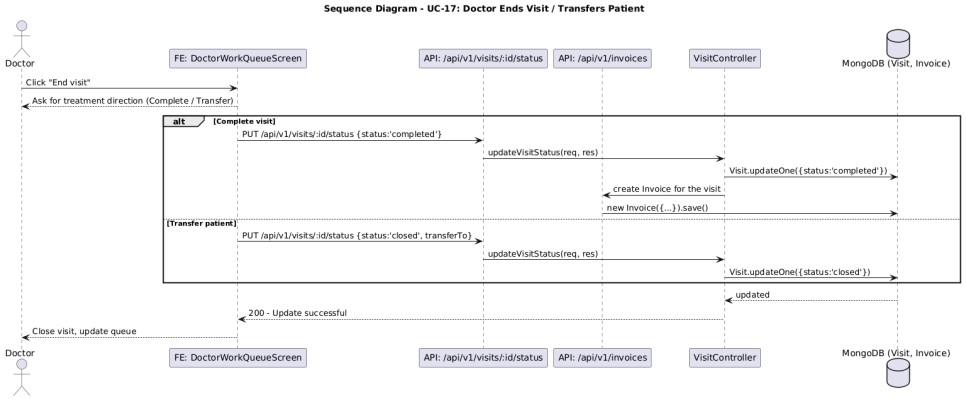

Figure 30: Sequence Diagram — UC-17

III.1.18 UC-18 — Confirm Invoice Payment

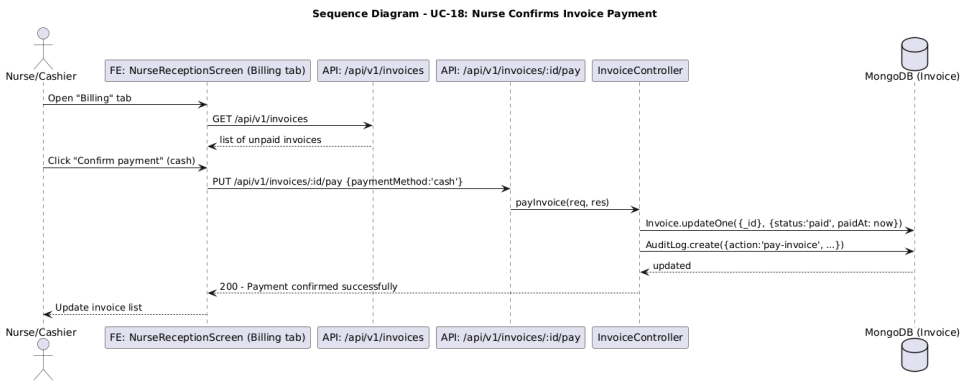

Figure 31: Sequence Diagram — UC-18

III.1.19 UC-19 — Assign Work Shifts

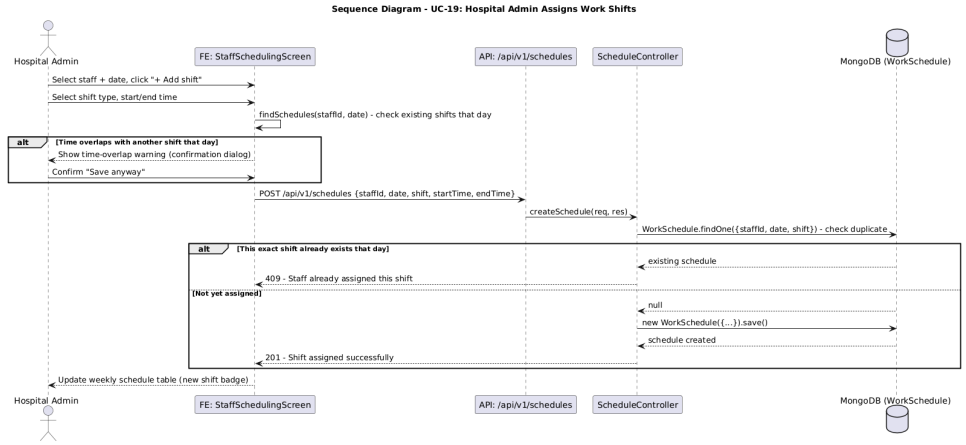

Figure 32: Sequence Diagram — UC-19

III.1.20 UC-20 — Request Shift Swap

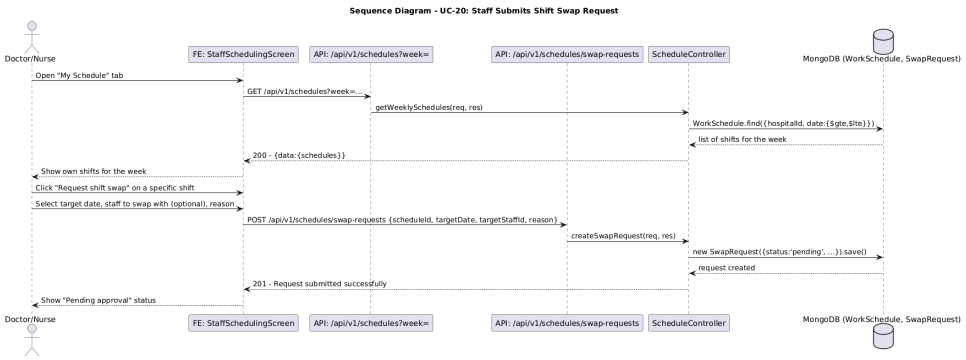

Figure 33: Sequence Diagram — UC-20

III.1.21 UC-21 — Review Swap Request

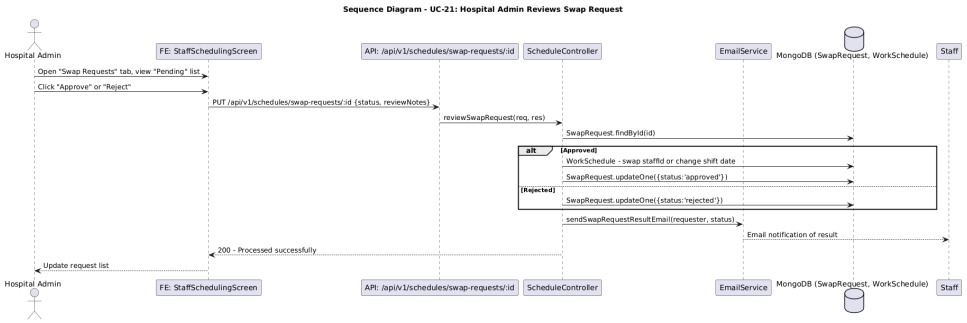

Figure 34: Sequence Diagram — UC-21

III.1.22 UC-22 — Update Drug Stock

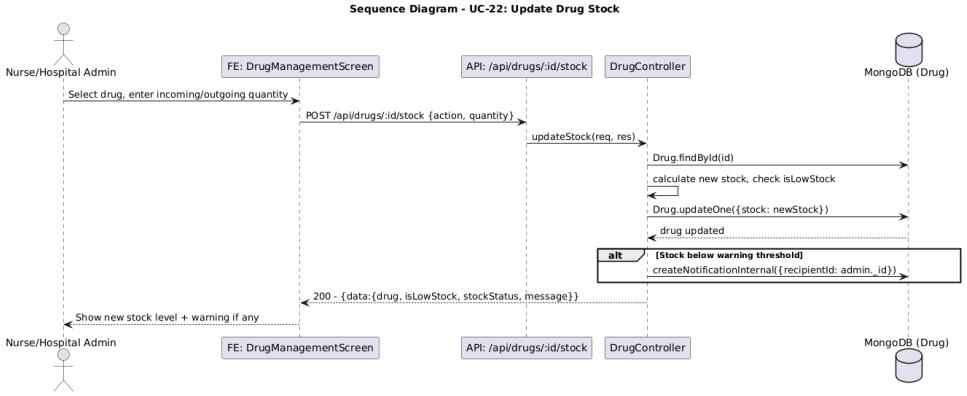

Figure 35: Sequence Diagram — UC-22

III.1.23 UC-23 — Provision Temp Hospital Account

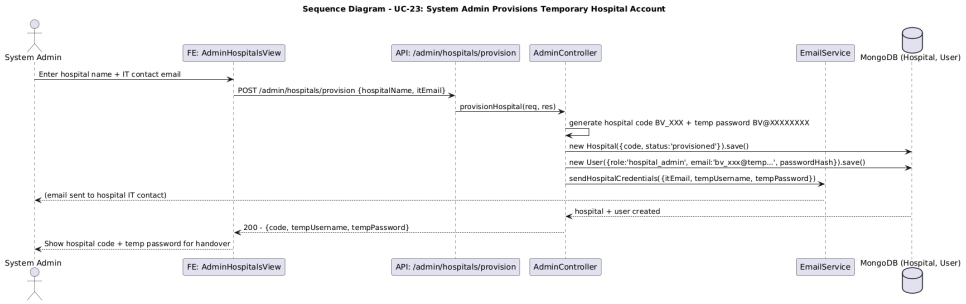

Figure 36: Sequence Diagram — UC-23

III.1.24 UC-24 — Submit Onboarding Information

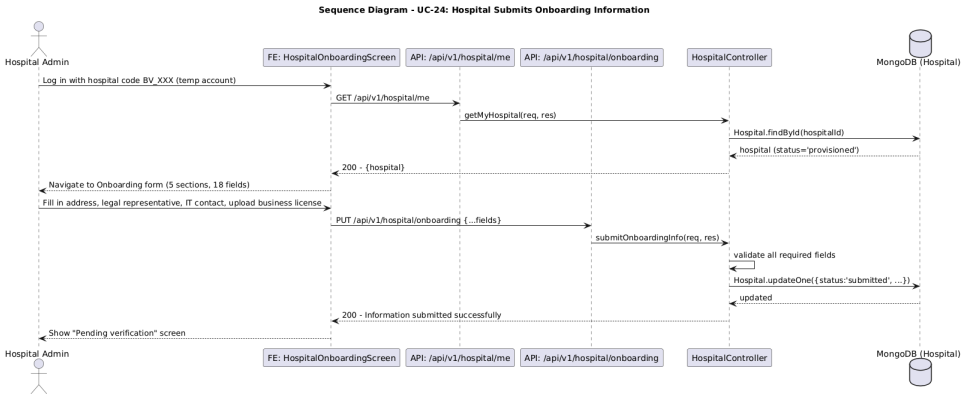

Figure 37: Sequence Diagram — UC-24

III.1.25 UC-25 — Activate Hospital

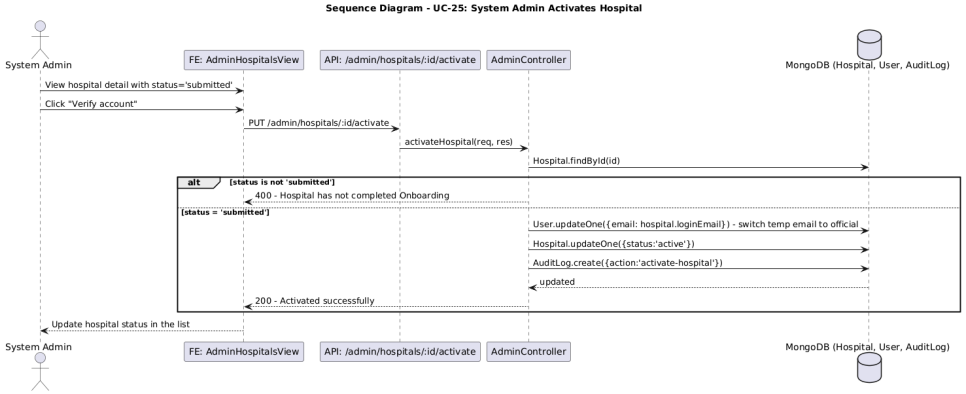

Figure 38: Sequence Diagram — UC-25

III.1.26 UC-26 — Lock / Unlock User Account

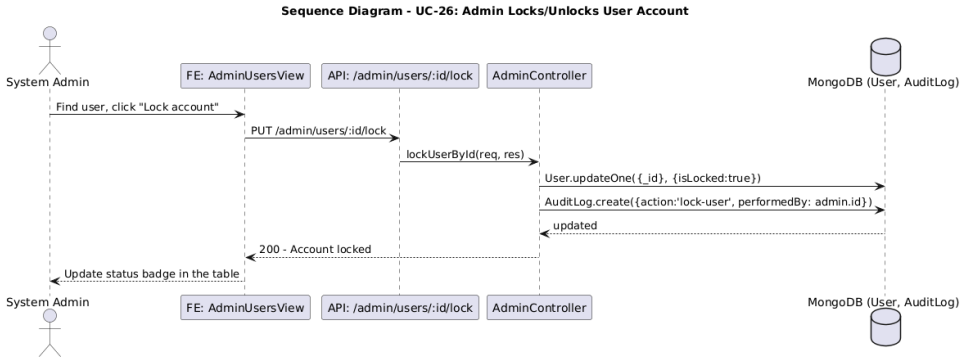

Figure 39: Sequence Diagram — UC-26

III.1.27 UC-27 — View System Statistics

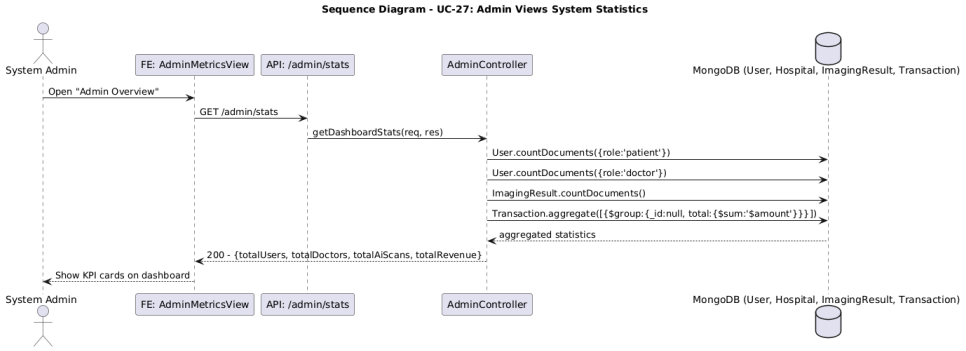

Figure 40: Sequence Diagram — UC-27

III.1.28 UC-28 — Verify Tenant Isolation

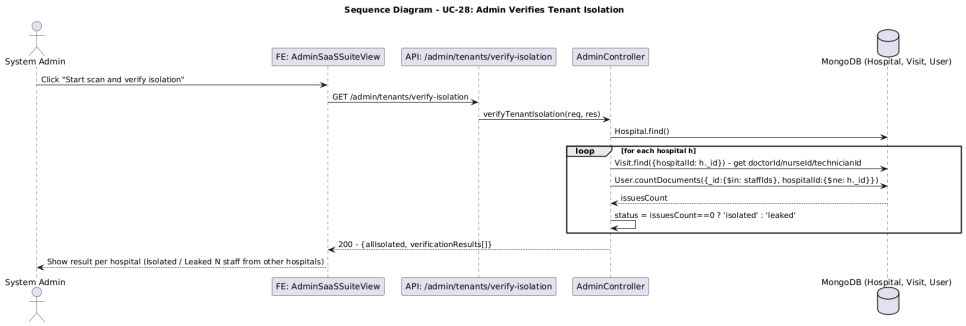

Figure 41: Sequence Diagram — UC-28

III.2 State Diagrams

III.2.1 Visit Lifecycle State Diagram

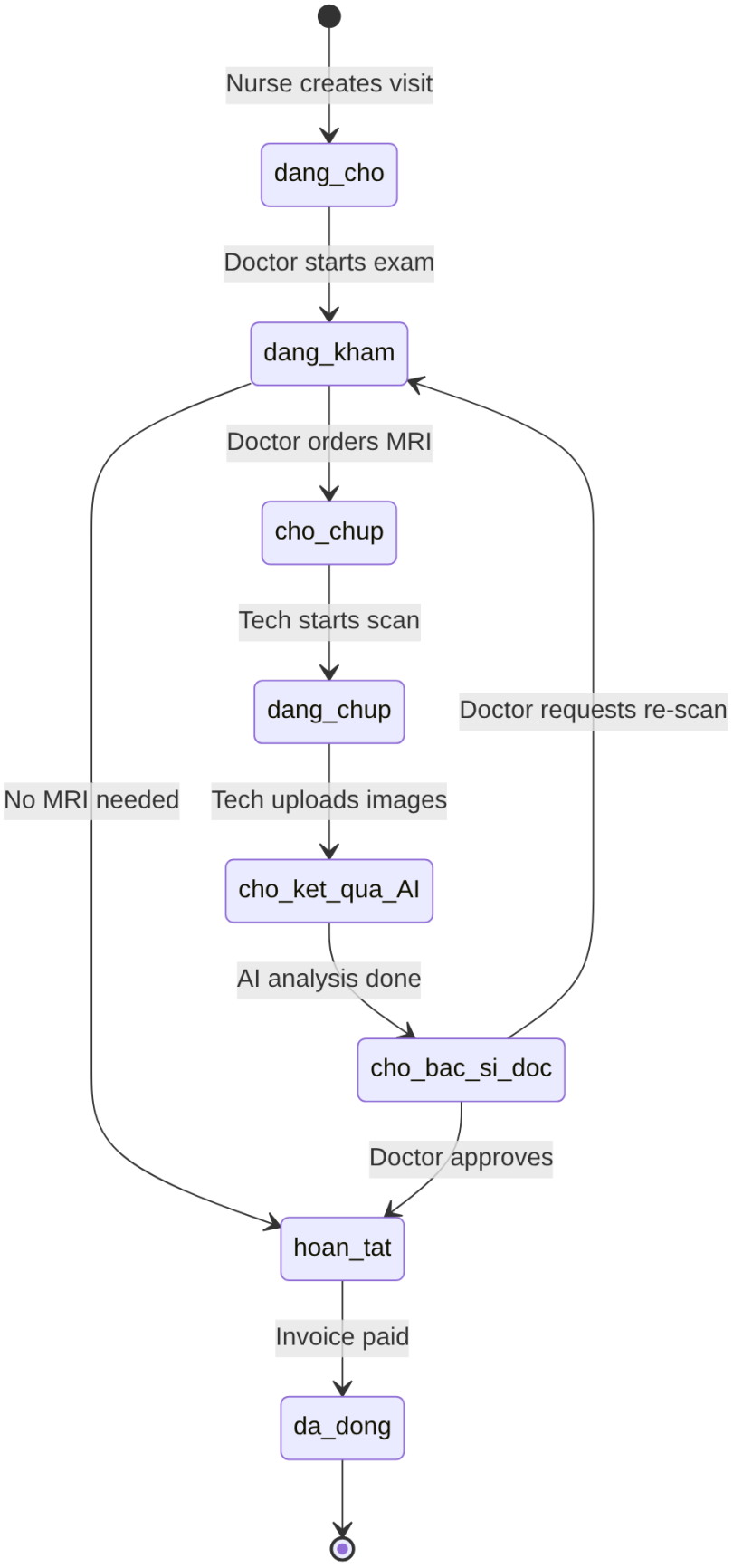

Figure 42: State Diagram — Visit Lifecycle

III.2.2 AI Prediction State Diagram

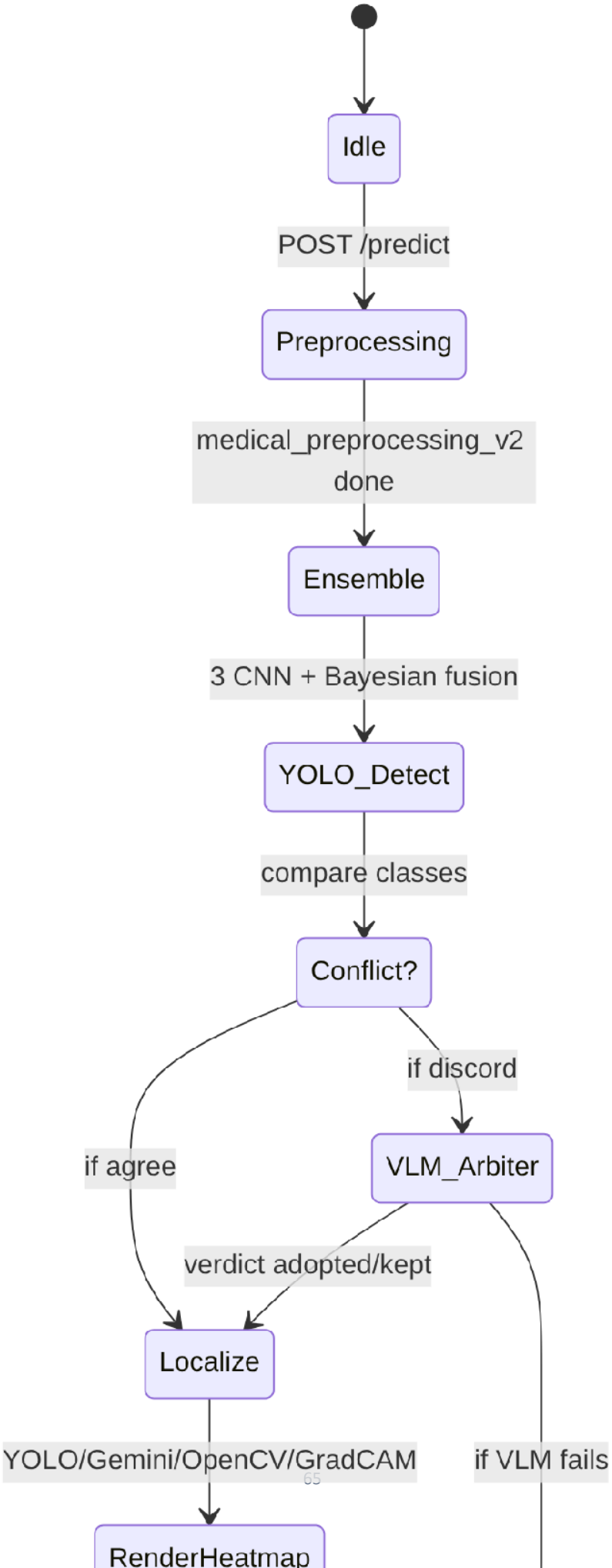
Idle
POST /predict
Preprocessing
medical_preprocessing_v2 done
Ensemble
3 CNN + Bayesian fusion
YOLO_Detect
compare classes
Conflict?
if discord
if agree
VLM_Arbiter
verdict adopted/kept
Localize
YOLO/Gemini/OpenCV/GradCAM
if VLM fails
RenderHeatmap

Figure 43: State Diagram — AI Prediction Pipeline

IV. Design Specification

IV.1 Integrated Communication Diagrams

The integrated component diagram below shows how all major system components collaborate end-to-end. The client apps (mobile + web admin) communicate with the Express backend over HTTPS/JSON with Bearer JWT. The backend orchestrates calls to the AI FastAPI server, Firebase Storage, Google Drive, Google Cloud Storage, Gemini API, and Firebase Cloud Messaging. MongoDB is the primary data store with tenant-scoped queries.

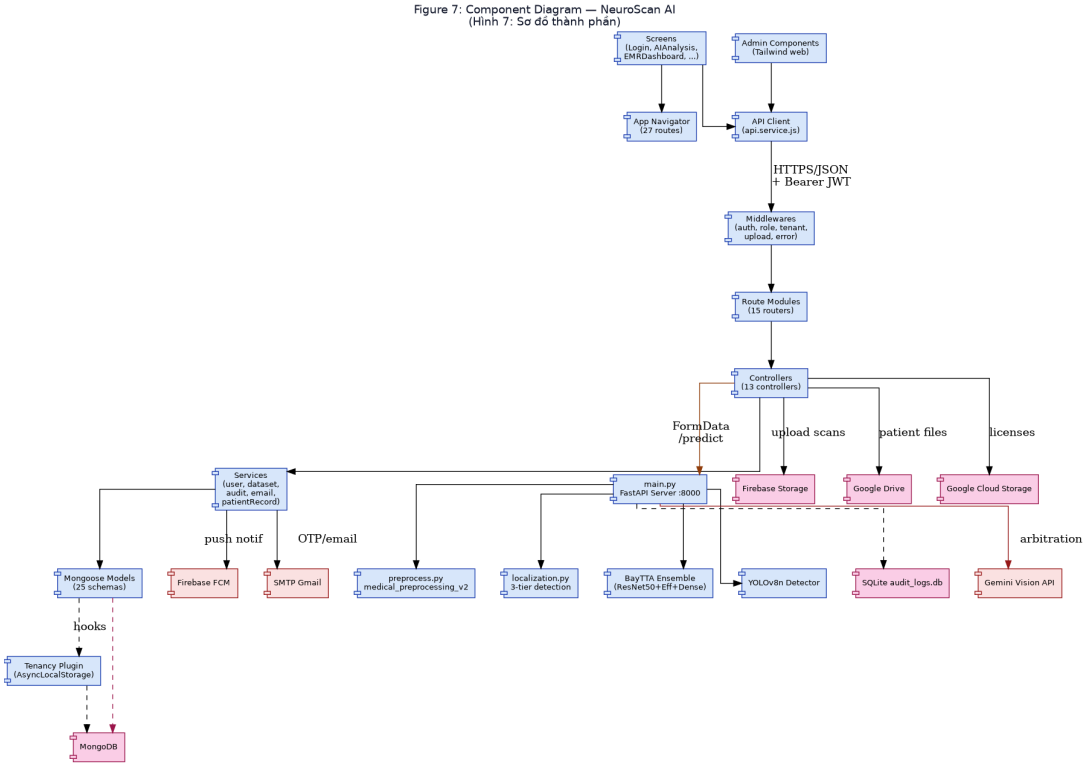

Figure 44: Integrated Component Diagram

IV.2 System High-Level Design

NeuroScan AI follows a 3-tier architecture: (1) Client tier with React Native (Expo) mobile app and Tailwind-based web admin portal; (2) Application tier with Node.js Express backend (port 3000) and Python FastAPI AI server (port 8000); (3) Data tier with MongoDB (sharded by hospitalId), SQLite (AI audit logs), and three cloud storage backends. NGINX acts as a reverse proxy handling TLS termination and request routing.

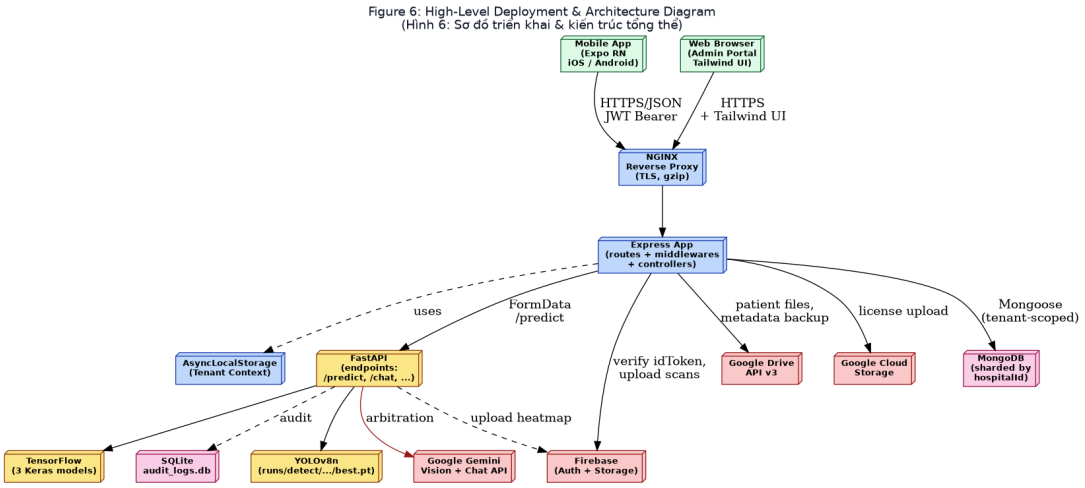

Figure 45: High-Level Deployment Diagram

IV.3 Component and Package Diagram

IV.3.1 Component Diagram

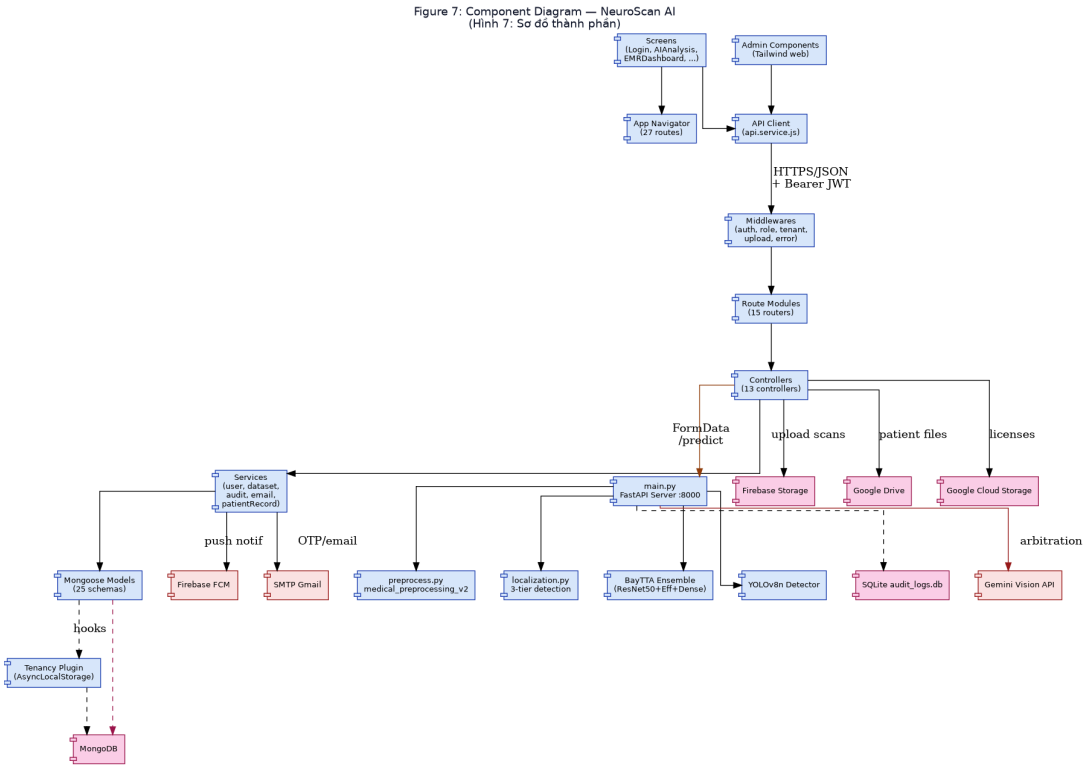

Figure 46: Component Diagram

IV.3.2 Package Diagram

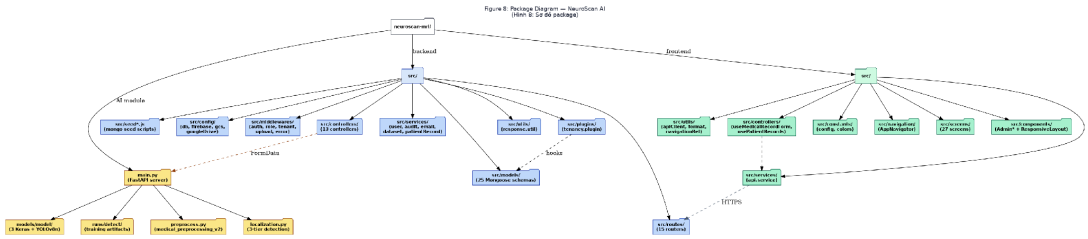

Figure 47: Package Diagram

Table 6: Package Descriptions

No	Package	Description
01	BE/src/config	Database (MongoDB), Firebase Admin SDK, Google Cloud Storage, Google Drive API v3

No	Package	Description
02	BE/src/middlewares	auth, role, tenant (AsyncLocalStorage), upload (multer), error (global handler)
03	BE/src/routes	15 routers + 2 new (reminder, billing)
04	BE/src/controllers	13 controllers + 2 new: billing.controller, reminder.controller
05	BE/src/services	5 services: user, audit, email, dataset, patientRecord
06	BE/src/models	27 Mongoose schemas + 2 new: MedicineReminder, Subscription
07	BE/src/plugins	tenancy.plugin — Mongoose plugin with AsyncLocalStorage
08	FE/src/screens	28 screens
09	FE/src/components	8 components (Admin* + ResponsiveLayout + PatientsView)
10	AI/main.py	FastAPI server with 8 endpoints + 4 LLM agents
11	AI/preprocess.py	medical_preprocessing_v2 — crop skull + CLAHE + 224×224
12	AI/localization.py	3-tier tumor localization (YOLO > Gemini > OpenCV > Grad-CAM)

IV.4 Class Diagrams

IV.4.1 Authentication & Multi-tenancy

This feature group covers user identity, hospital tenancy, and audit logging. The User class holds credentials, role, and hospitalId reference. The Hospital class holds SaaS subscription state and Drive folder structure. The AuthMiddleware class verifies JWT and activates tenant context; the TenantPlugin class reads the AsyncLocalStorage and injects hospitalId into Mongoose queries.

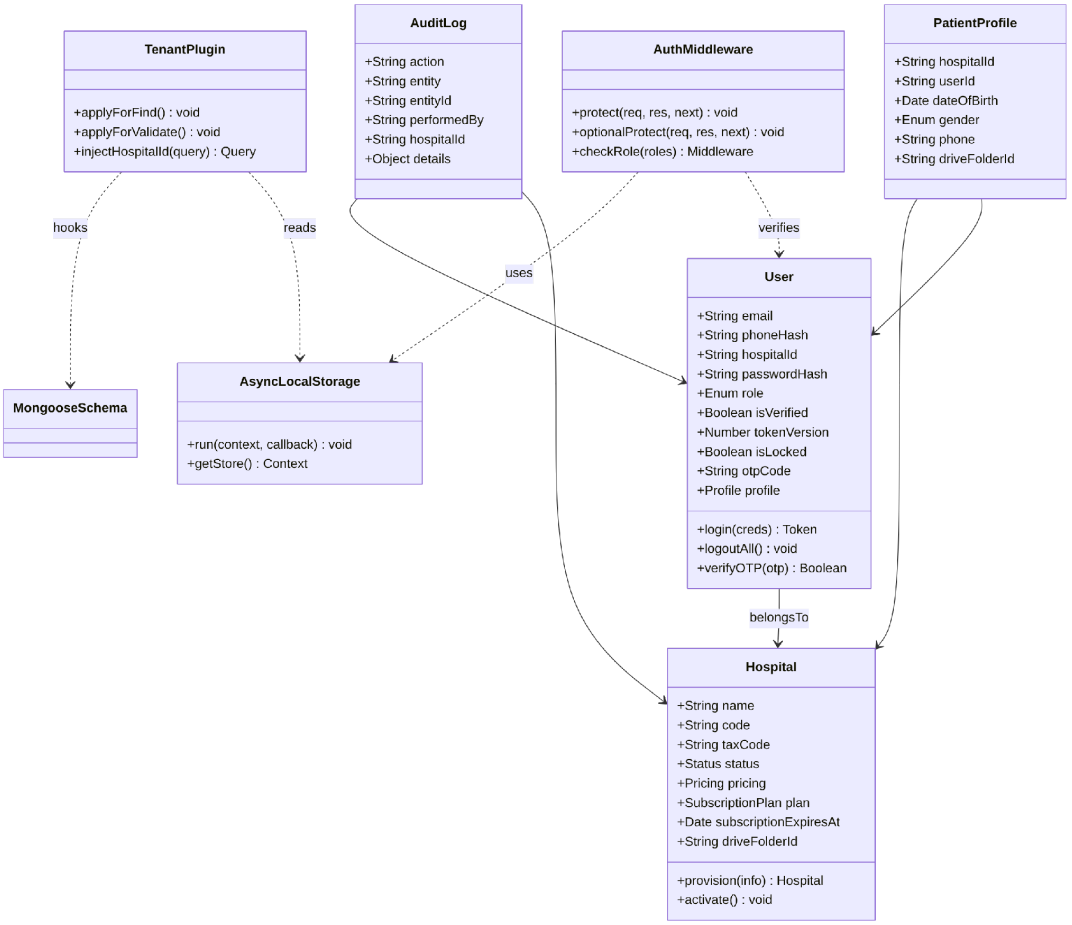

Figure 48: Class Diagram — Auth & Multi-tenancy

IV.4.2 Visit & AI Diagnosis Workflow

This feature group covers the visit lifecycle and the AI diagnosis pipeline. The FastAPIServer class uses a Strategy Pattern — the EnsembleStrategy abstract class has three concrete implementations (ResNet50Strategy w=0.8, EfficientNetStrategy w=0.1 T=1.3, DenseNetStrategy w=0.1 T=1.3). The YOLODetector and GeminiArbiter are collaborators invoked for cross-validation.

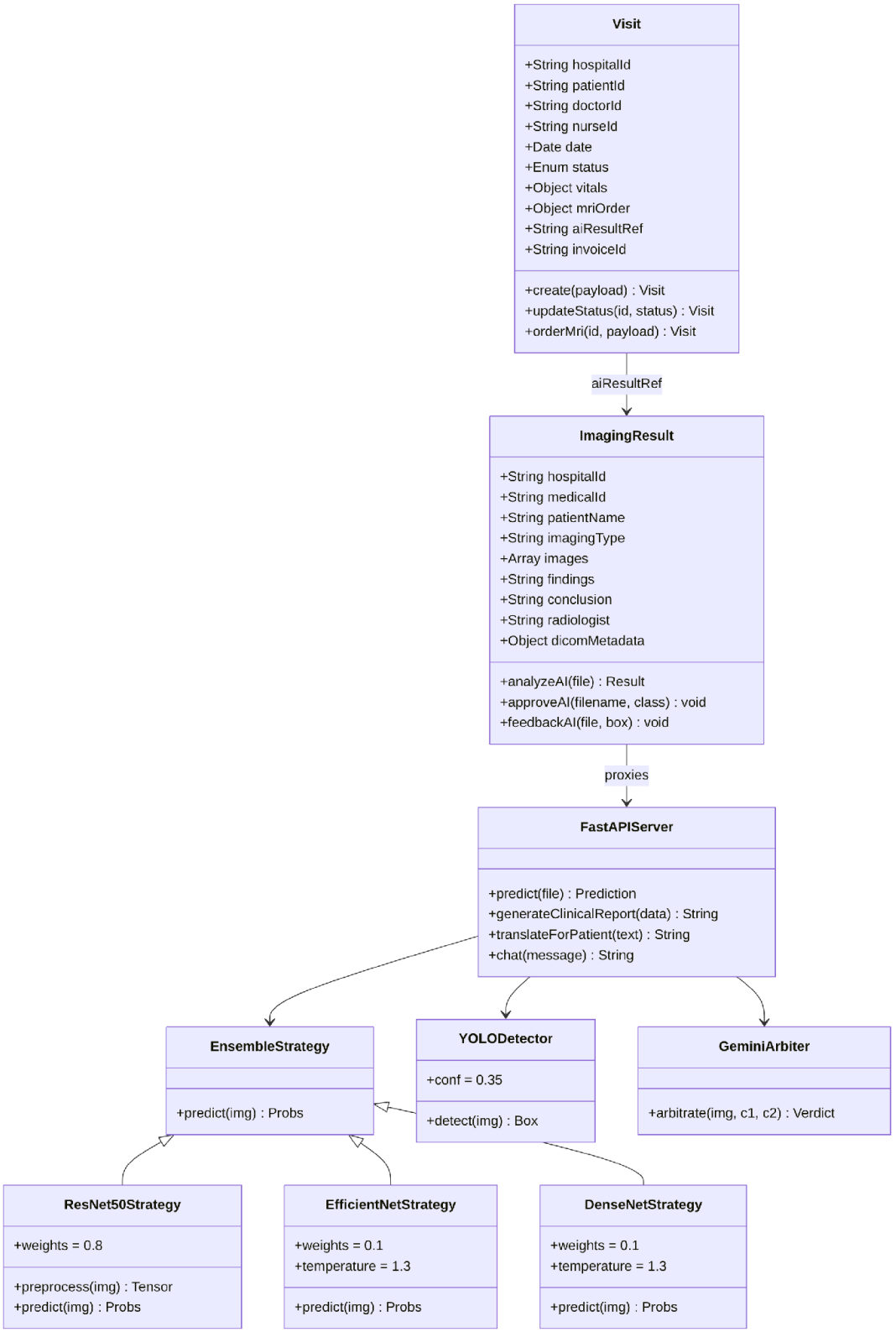

Figure 49: Class Diagram — Visit & AI

IV.4.3 Pharmacy & Drug Safety

This feature group covers drug inventory, prescriptions, lab orders, and the prescription safety checker. The DrugSafetyChecker class performs four safety checks (psychotropic, interactions, BMI, Gadolinium allergy).

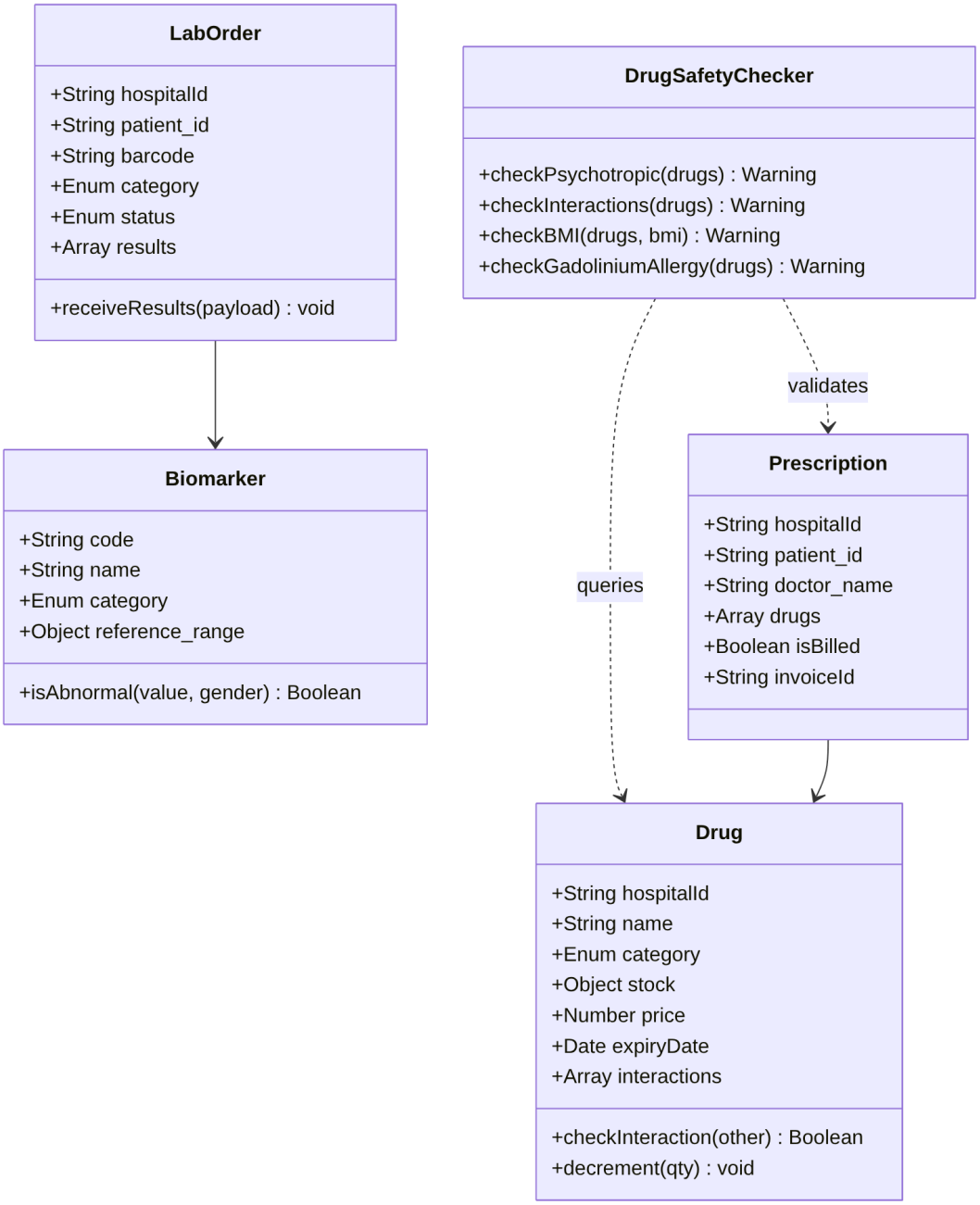

Figure 50: Class Diagram — Pharmacy

IV.5 Database Design

NeuroScan AI uses MongoDB as its primary data store. The database contains 29 collections, of which 14 are core clinical entities. All clinical collections are scoped by hospitalId via the tenancy plugin. The ER diagram (Figure 13) shows the 14 core entities and their relationships. The table below summarizes the schema design for these 14 collections, listing the most important fields, their data types, and key indexes.

Table 7: Database Schema — 14 Core Collections

Collection	Key Fields	Indexes
users	_id, email (unique), phoneHash (unique sparse), hospitalId, role, passwordHash, isVerified, tokenVersion, isLocked, profile	{email:1} unique, {phoneHash:1} unique sparse, {hospitalId:1, role:1}
hospitals	_id, name, code (unique), taxCode, status, pricing, subscriptionPlan, subscriptionExpiresAt, driveFolderId	{code:1} unique, {status:1}
patientprofiles	_id, hospitalId, userId (unique), dateOfBirth, gender, phone, driveFolderId	{userId:1} unique, {hospitalId:1}
visits	_id, hospitalId, patientId, doctorId, nurseId, technicianId, date, status, vitals, mriOrder, aiResultRef, invoiceId	{hospitalId:1, date:1}, {hospitalId:1, status:1}, {patientId:1}
medicalrecords	_id, hospitalId, patientId, diagnosis, treatmentPlan, status, signStatus, doctorInCharge, currentVersion	{hospitalId:1, patientId:1}, {signStatus:1}
imagingresults	_id, hospitalId, medicalId, patientName, imagingType, images, findings, conclusion, radiologist, dicomMetadata	{hospitalId:1, medicalId:1}, {hospitalId:1, imagingType:1}
drugs	_id, hospitalId, name, category, stock, price, expiryDate, interactions, isActive	{hospitalId:1, name:1}, {hospitalId:1, 'stock.quantity':1}

Collection	Key Fields	Indexes
prescriptions	_id, hospitalId, patient_id, doctor_name, diagnosis, drugs, isBilled, invoiceId	{hospitalId:1, patient_id:1}, {isBilled:1}
laborders	_id, hospitalId, patient_id, barcode (unique), category, status, results	{barcode:1} unique, {hospitalId:1, patient_id:1}
invoices	_id, hospitalId, patientId, visitId, items, totalAmount, status, paymentMethod, paidAt	{hospitalId:1, status:1}, {visitId:1}
notifications	_id, hospitalId, recipientId, type, title, message, isRead, createdAt	{recipientId:1, isRead:1, createdAt:-1}
auditlogs	_id, action, entity, entityId, performedBy, hospitalId, details, createdAt	{hospitalId:1, createdAt:-1}, {entity:1}
medicine_reminders (NEW)	_id, hospitalId, patientId, prescriptionId, drugName, dosageText, date, time, status, timesPerDay, dayIndex, slotIndex	{patientId:1, date:1, status:1} — supports UC-06, UC-10, UC-16
subscriptions (NEW)	_id, patientId (unique), plan, status, startedAt, expiresAt, amount, paymentProvider, providerTxnRef	{patientId:1, status:1} — supports UC-11

V. Implementation

V.1 Map Architecture to Project Structure

NeuroScan AI follows a Layered Architecture with three layers strictly separated across three independent codebases. The Presentation Layer is the React Native frontend (28 screens); the Application Layer is the Express backend (15 routers + 2 new, 13 controllers + 2 new, 27 models + 2 new); the AI Layer is the FastAPI server (8 endpoints). Layered Architecture was selected because it satisfies three non-functional requirements: NFR-03 (Multi-tenancy Isolation) — the tenancy plugin sits in the Application Layer and uniformly intercepts all data access; NFR-06 (Maintainability) — strict separation of concerns allows the AI team, backend team, and frontend team to work in parallel; NFR-08 (Auditability) — the AuditLog and EMRVersion mechanisms sit cleanly in the Service Layer where business state changes happen.

```
mri/                                  # Root repository (branch: Huyle_MRI/CT)
│
├── BE/                               # === Application Layer ===
│   ├── src/
│   │   ├── config/                   #   Infrastructure adapters
│   │   │   ├── db.js                 #     MongoDB connection
│   │   │   ├── firebase.js           #     Firebase Admin SDK
│   │   │   ├── gcs.js                #     Google Cloud Storage
│   │   │   └── googleDrive.js        #     Google Drive API v3
│   │   ├── middlewares/              #   Cross-cutting concerns
│   │   ├── routes/                   #   15 routers + 2 NEW (reminder,
billing)
│   │   ├── controllers/              #   13 controllers + 2 NEW (billing,
reminder)
│   │   ├── services/                 #   5 reusable services
│   │   ├── models/                   #   27 Mongoose schemas + 2 NEW
│   │   │   ├── medicineReminder.model.js  # NEW (UC-10, UC-16)
│   │   │   └── subscription.model.js      # NEW (UC-11)
│   │   ├── plugins/tenancy.plugin.js  #   Multi-tenancy via AsyncLocalStorage
│   │   └── index.js                  #   Express app entrypoint
│
├── FE/                               # === Presentation Layer ===
│   ├── src/
│   │   ├── navigation/AppNavigator.js #   28 routes
│   │   ├── screens/                  #   28 screens
│   │   ├── components/               #   8 components
│   │   └── services/api.service.js   #   HTTP client
│
└── MRIteam_team5/MRIteam/            # === AI Layer ===
    ├── main.py                       #   FastAPI server — 8 endpoints
    ├── preprocess.py                 #   medical_preprocessing_v2
    ├── localization.py               #   3-tier tumor localization
    └── runs/detect/mri_tumor_det/    #   YOLOv8n training artifacts
(mAP50=0.961)
```

V.2 Map Class Diagram to Code — Design Patterns

V.2.1 Pattern 1: Multi-tenancy via AsyncLocalStorage + Mongoose Plugin

The tenant.middleware.js defines an AsyncLocalStorage that holds the current request's hospitalId. The auth.middleware.js runs the request handler inside tenantStorage.run({hospitalId}). The tenancy.plugin.js registers Mongoose pre-hooks on find/findOne/update/delete/validate that read this context and inject hospitalId into the query (for reads) or set it on the document (for writes). This ensures every clinical query is automatically scoped — there is no way for a controller to accidentally read another hospital's data.

```
// BE/src/middlewares/tenant.middleware.js
import { AsyncLocalStorage } from "node:async_hooks";
export const tenantStorage = new AsyncLocalStorage();
export function getHospitalIdContext() {
  return tenantStorage.getStore()?.hospitalId || null;
}

// BE/src/middlewares/auth.middleware.js
export const protect = async (req, res, next) => {
  // ... verify JWT, fetch user ...
  return tenantStorage.run({ hospitalId: user.hospitalId }, () => next());
};

// BE/src/plugins/tenancy.plugin.js
export function applyTenancyPlugin(schema) {
  ["find", "findOne", "countDocuments", "findOneAndUpdate",
   "updateOne", "updateMany", "findOneAndDelete", "deleteOne", "deleteMany"
  ].forEach((op) => {
    schema.pre(op, function (next) {
      const ctx = tenantStorage.getStore();
      if (ctx?.hospitalId && !this.getQuery().hospitalId) {
        this.where({ hospitalId: ctx.hospitalId });
      }
      next();
    });
  });
  schema.pre("validate", function (next) {
    const ctx = tenantStorage.getStore();
    if (ctx?.hospitalId && !this.hospitalId) {
      this.hospitalId = ctx.hospitalId;
    }
    next();
  });
}
```

V.2.2 Pattern 2: Strategy Pattern — BayTTA Ensemble

The AI ensemble uses the Strategy Pattern: an abstract EnsembleStrategy defines the predict interface, and three concrete strategies (ResNet50, EfficientNet, DenseNet) implement it with their own preprocessing function, weight, and temperature scaling. The FastAPIServer context holds a list of strategies and aggregates their predictions via entropy-weighted Bayesian fusion. This design allows adding a fourth CNN (e.g., ConvNeXt) by simply implementing a new strategy — no changes to the prediction orchestration code.

```
# MRIteam/main.py — Strategy Pattern in BayTTA Ensemble
from tensorflow.keras.applications import resnet_v2, efficientnet_v2, densenet
res_prep = resnet_v2.preprocess_input
eff_prep = efficientnet_v2.preprocess_input
den_prep = densenet.preprocess_input

WEIGHTS = { "resnet": 0.8, "efficientnet": 0.1, "densenet": 0.1 }
TEMPERATURE = 1.3

def apply_temperature_scaling(probs, T=1.3):
    logits = np.log(probs + 1e-8)
    scaled = logits / T
    return np.exp(scaled) / np.sum(np.exp(scaled), axis=-1, keepdims=True)

def baytta_ensemble(img_array):
    views = [img_array, np.flip(img_array, axis=2)]  # original + horizontal
flip
    view_preds = []
    for view in views:
        p_res = model.predict(res_prep(view[None, ...].astype("float32")),
verbose=0)
        p_eff = apply_temperature_scaling(
            efficientnet_model.predict(eff_prep(view[None,
...].astype("float32")), verbose=0), T=TEMPERATURE)
        p_den = apply_temperature_scaling(
            densenet_model.predict(den_prep(view[None,
...].astype("float32")), verbose=0), T=TEMPERATURE)
        ensemble = (WEIGHTS["resnet"] * p_res +
                    WEIGHTS["efficientnet"] * p_eff +
                    WEIGHTS["densenet"] * p_den)
        entropy = -np.sum(ensemble * np.log(ensemble + 1e-8), axis=-1)
        view_preds.append((ensemble, np.exp(-entropy[0])))
    weights = np.array([v[1] for v in view_preds])
    weights = weights / np.sum(weights)
    final_pred = sum(w * v[0] for w, v in zip(weights, view_preds))
    return final_pred  # shape (1, 4)
```

V.2.3 Pattern 3: Auto-Generate Reminders (UC-16 / BR-06)

When a doctor creates a prescription, the system automatically generates MedicineReminder documents for each drug × each day in the duration × each time slot defined by BR-06. This is implemented as a helper function called from the patient controller after the prescription is saved.

```
// BE/src/controllers/reminder.controller.js
// BR-06: Reminder Time Slots
export const REMINDER_TIME_SLOTS = {
  1: ["08:00"],
  2: ["08:00", "20:00"],
  3: ["08:00", "13:00", "20:00"],
  4: ["08:00", "12:00", "17:00", "21:00"],
};

export const generateRemindersForPrescription = async (prescription,
hospitalId) => {
  const reminders = [];
  for (const drug of prescription.drugs) {
    const timesPerDay = Math.min(Math.max(drug.timesPerDay || 1, 1), 4);
    const durationDays = Math.min(Math.max(drug.durationDays || 7, 1), 90);
    const slots = REMINDER_TIME_SLOTS[timesPerDay];
    const startDate = new Date(prescription.createdAt || Date.now());
    for (let dayIndex = 0; dayIndex < durationDays; dayIndex++) {
      const date = new Date(startDate);
      date.setDate(date.getDate() + dayIndex);
      const dateStr = date.toISOString().split("T")[0];
      for (let slotIndex = 0; slotIndex < slots.length; slotIndex++) {
        reminders.push({
          hospitalId, patientId: prescription.patient_id,
          prescriptionId: prescription._id,
          drugName: drug.name,
          dosageText: `${drug.quantity} ${drug.unit || "tablet"} ${drug.usage
|| ""}`.trim(),
          date: dateStr, time: slots[slotIndex], status: "pending",
          timesPerDay, dayIndex, slotIndex,
        });
      }
    }
  }
  if (reminders.length > 0) {
    await MedicineReminder.insertMany(reminders);
  }
  return reminders;
};

// BE/src/controllers/patient.controller.js — addPatientPrescription
const newItem = new Prescription({ patient_id, doctor_name, diagnosis, drugs,
note });
await newItem.save();
// UC-16: Auto-generate MedicineReminder per drug x day x time slot (BR-06)
try {
  const { generateRemindersForPrescription } = await
import("./reminder.controller.js");
  await generateRemindersForPrescription(newItem, req.user?.hospitalId);
} catch (remErr) {
  console.error("Failed to generate reminders:", remErr.message);
}
```

VI. Applying Alternative Architecture Patterns

This section redesigns parts of NeuroScan AI using three alternative architectural patterns: Service-Oriented Architecture (SOA), the Service Discovery Pattern (within SOA/microservices), and Event-Driven Architecture. For each, we identify the non-functional requirement(s) not fully supported by the current monolithic-with-AI-sidecar architecture, propose a solution, and provide supporting diagrams.

VI.1 Service-Oriented Architecture (SOA)

Problem Identification: The current architecture is essentially a modular monolith — the Express backend contains 15 route modules, 13 controllers, and 5 services in a single deployable unit. While internal modularity is good (NFR-06 Maintainability is satisfied), two non-functional requirements are not fully supported: NFR-04 (Scalability) — different modules have different load profiles (the imaging module is I/O-bound calling FastAPI, while the auth module is CPU-bound on bcrypt hashing); scaling them together wastes resources. NFR-05 (Availability) — a crash in the drug module takes down the entire backend, including the imaging module that has nothing to do with drugs.

Solution: Reorganize the backend into seven independent services, each owning its own data store (or sharded subset) and communicating via HTTP/JSON or gRPC. The API Gateway (NGINX/Kong) handles cross-cutting concerns: TLS termination, JWT verification, rate limiting, request routing. Each service can be deployed, scaled, and versioned independently: Auth Service (users + otps), Clinical Service (visits + medicalrecords + EMR), Imaging Service (imagingresults + proxy to AI), Drug Service (drugs + prescriptions), Notification Service (notifications + FCM), Storage Service (Firebase/Drive/GCS abstraction), and AI Service (existing FastAPI server, now first-class).

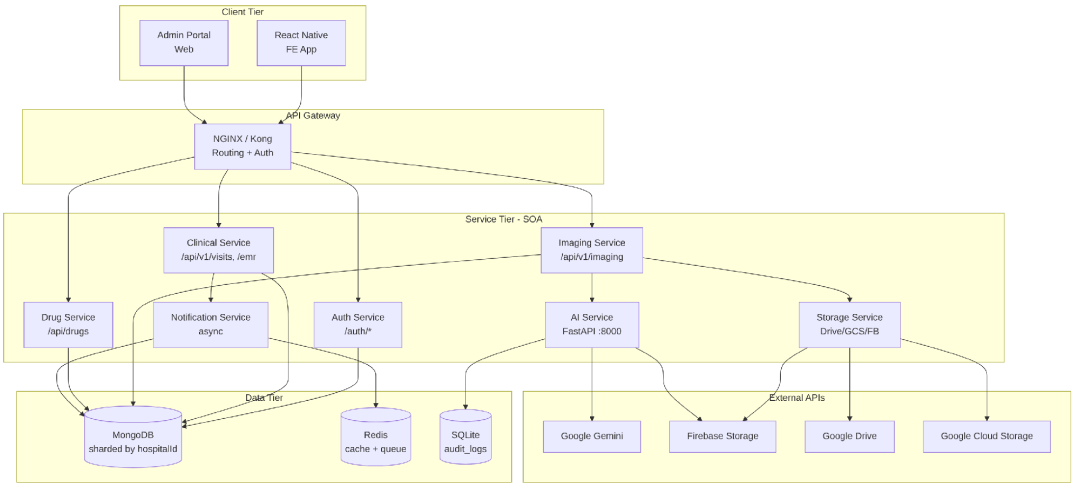

Figure 51: SOA Deployment Diagram

VI.2 Service Discovery Pattern

Problem & Requirement: With the SOA redesign above, NFR-04 (Scalability) is partially addressed but a new problem emerges — when the Clinical Service is scaled to 3 instances (CS1, CS2, CS3) and the AI Service to 2 instances (AIS1 on CPU, AIS2 on GPU), how does the API Gateway know which instance to route a request to? Hardcoding IPs/ports breaks down when instances are dynamically added/removed (autoscaling, crash recovery, rolling deployments). We need NFR-04b: Dynamic Service Discovery — the ability to locate service instances at runtime without hardcoded configuration.

Solution: Integrate a Service Discovery Pattern using a registry (Consul, Eureka, or Kubernetes DNS). Each service instance registers itself with the registry on startup (with its IP, port, health check endpoint) and sends periodic heartbeats. The API Gateway queries the registry for healthy instances of a given service name, then uses a load balancer (round-robin, least-connections, or weighted) to pick one instance per request. For the AI Service specifically, the registry can store metadata — e.g., AIS1 has tag {gpu:false}, AIS2 has tag {gpu:true} — enabling metadata-aware routing.

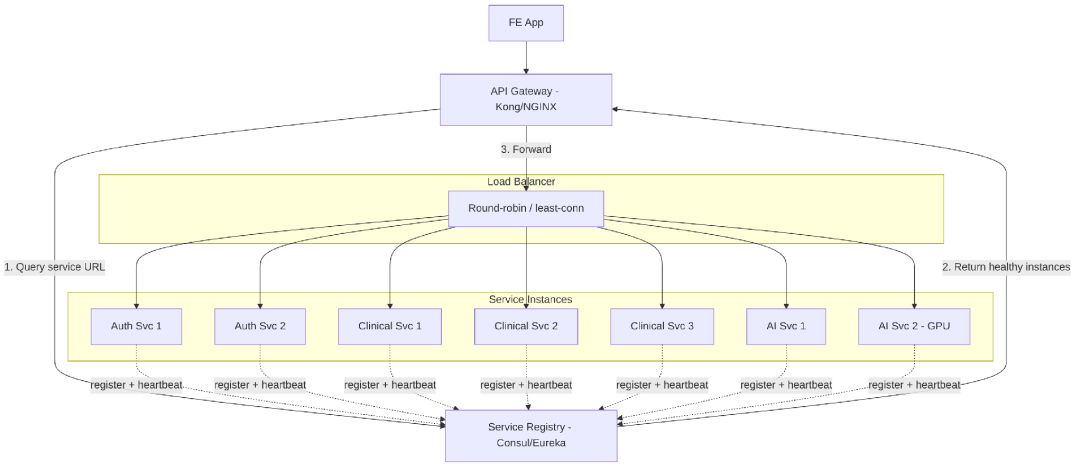

Figure 52: Service Discovery Pattern Deployment

VI.3 Event-Driven Architecture

Problem & Requirement: In the current architecture, cross-cutting concerns like notifications, audit logging, and dashboard analytics are implemented synchronously — when the Clinical Service creates a Visit, it directly calls Notification Service (which writes to DB) and Audit Service (which writes AuditLog) within the same request. This couples the request latency to these side-effects (NFR-01 Performance not fully met) and creates a failure cascade — if Notification Service is slow, the Visit creation API is slow too. We need NFR-01b: Asynchronous Side-Effects — side-effects should not block the main request.

Solution: Introduce a Message Broker (Apache Kafka or RabbitMQ) between producers and consumers. Producers (Clinical, Imaging, Invoice, Drug services) emit events to topics; consumers (Notification, AI, Billing, Stock, Audit, Analytics services) subscribe to topics they care about. The producer's request returns immediately after persisting the main state change + publishing the event — side-effects happen asynchronously. Benefits: (1) Decoupled latency — Visit creation returns in 50 ms instead of 200+ ms; (2) Fault tolerance — if Audit Service is down, events are buffered in Kafka; (3) Extensibility — adding a new consumer requires zero changes to producers. Trade-offs: (1) Eventual consistency — AuditLog appears a few seconds after the operation; (2) Operational complexity — Kafka cluster management; (3) At-least-once delivery semantics — consumers must be idempotent.

Table 8: Event Topics & Consumers

Topic	Producer	Consumers	Purpose
visit.created	Clinical Service	Notification, Audit, Analytics	Trigger push notification; write audit log; update dashboard
mri.ordered	Clinical Service	AI Service, Notification	Pre-warm AI model; notify technician
ai.result.ready	Imaging Service	Notification, Audit	Push to doctor; write audit log
invoice.paid	Invoice Service	Stock, Audit, Analytics	Decrement drug stock; mark prescription billed; update revenue
drug.stock.low	Drug Service	Notification, Stock	Alert hospital_admin; trigger reorder

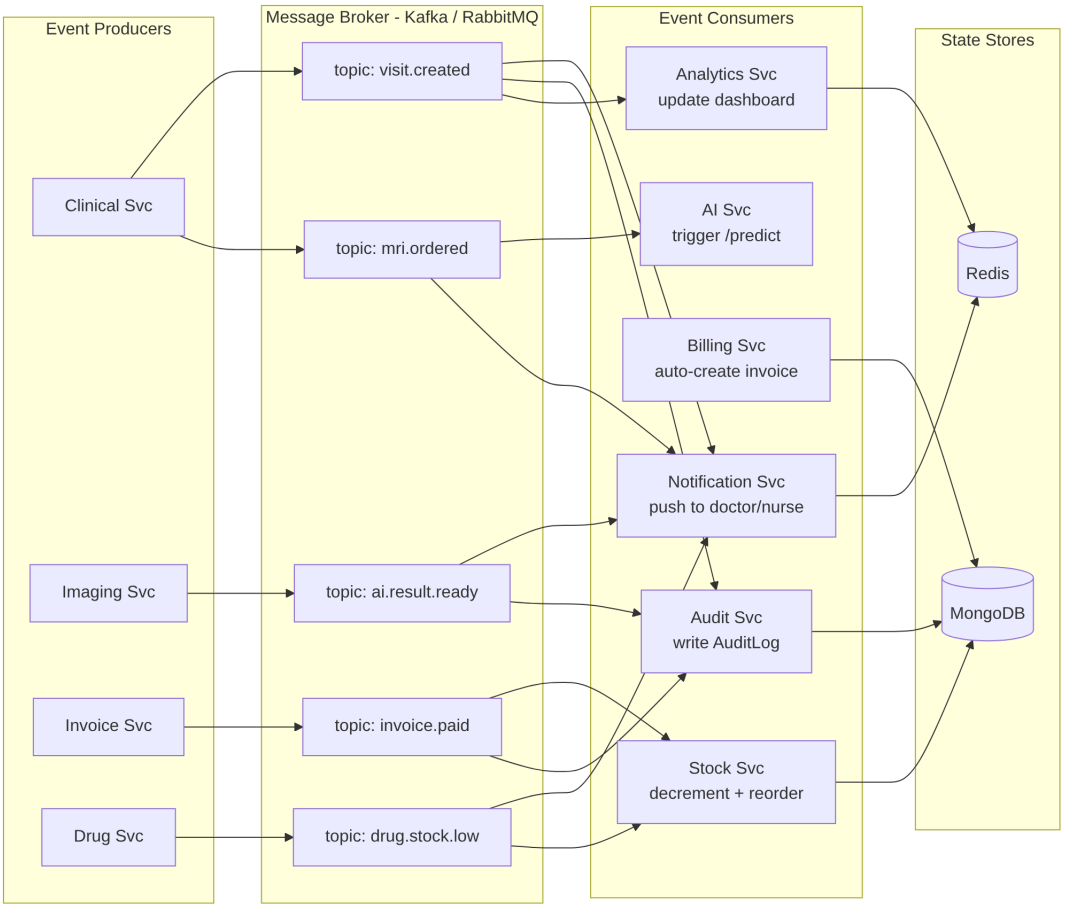

Figure 53: Event-Driven Architecture Diagram